



ISGA CASABLANCA

3CI — Big Data et Intelligence Artificielle

PROJET DE FIN D'ÉTUDES

Conception et réalisation de DataWatch

Plateforme SaaS multi-tenant de surveillance de la qualité des données,
enrichie par l'IA pour l'explication des incidents

RÉALISÉ PAR

Mounir Gaiby

ENCADRÉ PAR

Dr. HANINE MOHAMED

FILIÈRE

3CI — Big Data et Intelligence Artificielle

Année universitaire 2025–2026

I. Dédicace

Je dédie ce travail à ma famille, à mes proches et à toutes les personnes qui ont soutenu mon parcours académique.

II. Remerciements

Je tiens à exprimer ma reconnaissance à Dr. HANINE MOHAMED pour son encadrement, ses orientations méthodologiques et ses retours tout au long de ce projet. Mes remerciements s'adressent également aux enseignants de l'ISGA Casablanca, dont les enseignements ont constitué le socle technique et scientifique de ce travail.

Je remercie enfin ma famille et mes collègues pour leur soutien, leurs encouragements et leur disponibilité. Leur confiance a été déterminante dans l'aboutissement de ce projet de fin d'études.

III. Résumé

La qualité des données est un enjeu critique pour les organisations qui prennent des décisions à partir d'entrepôts, de bases opérationnelles et de flux analytiques. Pourtant, les défauts de fraîcheur, les ruptures de schéma, les pics de valeurs nulles ou les anomalies de volume sont souvent détectés tardivement, sans contexte exploitable par les équipes.

Ce projet présente DataWatch, une plateforme SaaS multi-tenant de surveillance de la qualité des données. Son architecture répond aux quatre tensions du Big Data : le volume est condensé par des agrégats calculés sur la source, la variété passe par des contrats de connecteurs, la vélocité par une exécution Redis/Celery et la véracité par des profils horodatés. La détection associe règles métier, z-score, Isolation Forest et STL avant de créer des incidents priorisés. Une narration par grand modèle de langage propose ensuite un résumé, des hypothèses et des actions de diagnostic, sans retirer la décision à l'opérateur.

La réalisation s'appuie sur FastAPI, PostgreSQL, Redis, Celery, React et Docker. Une validation locale reproductible a couvert les parcours de surveillance, d'alerte et de moniteurs, ainsi qu'un petit prototype de gouvernance IA en mode observation. Les résultats montrent la faisabilité d'une chaîne de détection, d'explication et d'alerte. Ils ne transforment ni la gouvernance expérimentale ni les preuves locales en garantie de production.

Mots-clés : *qualité des données, observabilité, SaaS, multi-tenancy, détection d'anomalies, IA générative, gouvernance IA.*

IV. Abstract

Data quality is essential for organisations that rely on warehouses, operational databases and analytical flows. Freshness breaches, schema changes, null-value spikes and volume anomalies are often detected too late and without enough context for effective investigation.

This project presents DataWatch, a multi-tenant SaaS platform for data-quality monitoring. Its architecture addresses four recurring data-system pressures: volume through source-side aggregation, variety through connector contracts, velocity through Redis/Celery execution, and veracity through timestamped profiles. Detection combines business rules, z-scores, Isolation Forest and STL before prioritised incidents are created. A large-language-model layer then supplies a structured summary, plausible hypotheses and investigation actions while keeping the operator in control.

The implementation uses FastAPI, PostgreSQL, Redis, Celery, React and Docker. A reproducible local validation covered monitoring, alerts, monitors and AI-governance flows. The work demonstrates an end-to-end detection, explanation and alerting chain while explicitly separating local evidence from production guarantees.

Keywords : *data quality, observability, SaaS, multi-tenancy, anomaly detection, generative AI, AI governance.*

V. Table des matières

I. Dédicace.....	i
II. Remerciements.....	ii
III. Résumé.....	iii
IV. Abstract.....	iv
V. Table des matières.....	v
VI. Liste des figures.....	vii
VII. Liste des tableaux.....	ix
VIII. Liste des abréviations.....	xi
INTRODUCTION GÉNÉRALE.....	1
1.1 Contexte et motivation.....	1
1.2 Problématique et démarche.....	1
1.3 Résultats attendus.....	2
1.4 Organisation du rapport.....	2
CHAPITRE 1 — CADRE GÉNÉRAL ET CONDUITE DU PROJET.....	3
1.1 Introduction.....	3
1.2 Présentation de l'organisme d'accueil.....	3
1.2.1 Présentation de l'ISGA.....	3
1.2.2 Environnement pédagogique du projet.....	3
1.3 Présentation du projet.....	4
1.3.1 Contexte métier.....	4
1.3.2 Périmètre fonctionnel.....	4
1.3.3 Utilisateurs cibles.....	4
1.3.4 Problématique.....	4
1.3.5 Solution proposée.....	5
1.4 Méthodologie de travail.....	5
1.4.1 Méthode Agile.....	5
1.4.2 Organisation en sprints.....	6
1.4.3 Diagramme de Gantt et jalons.....	6
1.5 Conclusion.....	7
CHAPITRE 2 — ÉTAT DE L'ART ET POSITIONNEMENT.....	8
2.1 Introduction.....	8
2.2 Approches existantes.....	8
2.2.1 Contrôles déclaratifs.....	8
2.2.2 Détection statistique et apprentissage non supervisé.....	8
2.2.3 Observabilité et gestion d'incidents.....	8
2.3 Originalité de la solution.....	9
2.4 Conclusion.....	10
CHAPITRE 3 — ANALYSE, CONCEPTION ET ARCHITECTURE.....	11
3.1 Introduction.....	11
3.2 Analyse du projet.....	11
3.2.1 Objectifs de la solution.....	11
3.2.2 Besoins fonctionnels.....	11
3.2.3 Besoins non fonctionnels.....	12
3.3 Conception de la solution.....	13
3.3.1 Acteurs et cas d'utilisation.....	13
3.3.2 Diagramme de séquence.....	15
3.3.3 Diagramme de classes.....	18
3.4 Architecture proposée.....	21
3.4.1 Vue logique en couches.....	21
3.4.2 Exécution asynchrone.....	21
3.4.3 Architecture des connecteurs.....	21
3.5 Modèle de données et sécurité.....	24
3.5.1 Isolation multi-tenant.....	24
3.5.2 Protection des secrets.....	24
3.5.3 Intégrité et conservation.....	24
3.5.4 Structure relationnelle complète.....	24
3.6 Gouvernance de la couche IA.....	29
3.7 Conclusion.....	29
CHAPITRE 4 — IMPLÉMENTATION TECHNIQUE ET TESTS.....	30
4.1 Introduction.....	30
4.2 Workflow proposé.....	30

4.2.1 Enregistrement et planification.....	30
4.2.2 Profilage et détection.....	30
4.2.3 Incident, narration et alerte.....	30
4.3 Technologies utilisées.....	31
4.4 Présentation des interfaces.....	32
4.4.1 Vue Opérations.....	32
4.4.2 Détail de l'incident P1.....	34
4.4.3 Détail de la table surveillée.....	37
4.4.4 Alertes et gouvernance IA.....	41
4.4.4.1 Rapports, équipes et astreintes.....	41
4.4.4.2 Sources, connecteurs et notifications.....	44
4.4.4.3 Registre de gouvernance IA.....	48
4.4.4.4 Limites du prototype de gouvernance IA.....	51
4.4.4.5 Portail staff.....	52
4.4.5 Règles de présentation des captures.....	55
4.5 Tests et validation.....	55
4.5.1 Stratégie de test.....	55
4.5.2 Scénarios fonctionnels.....	56
4.5.3 Résultats et interprétation.....	56
4.6 Discussion des limites.....	57
4.7 Conclusion.....	58
CONCLUSION GÉNÉRALE ET PERSPECTIVES.....	59
Cadre du travail.....	59
Synthèse des apports.....	59
Perspectives.....	59
RÉFÉRENCES.....	60
ANNEXES.....	61
Annexe A — Procédure de démonstration.....	61
Annexe B — Indicateurs à interpréter avec prudence.....	61
Annexe C — Guide de captures d'écran pour la soutenance.....	61
C.1 Tableau de bord Opérations.....	61
C.2 Détail de l'incident orders.....	61
C.3 Routage d'alerte et gouvernance IA.....	61
Annexe D — Dictionnaire du modèle de données.....	62
Annexe E — État du jeu de démonstration.....	79
Annexe F — Commandes de reproduction.....	82

VI. Liste des figures

CHAPITRE 1 — CADRE GÉNÉRAL

Figure 1.1 — Planification du PFE du 1er juin au 31 août et continuité du produit.....	7
--	---

CHAPITRE 3 — ANALYSE ET CONCEPTION

Figure 3.1 — Vue globale des acteurs et domaines fonctionnels.....	13
Figure 3.2 — Cas d'utilisation du workspace client.....	14
Figure 3.3 — Cas d'utilisation du portail staff.....	15
Figure 3.4 — Séquence de connexion et d'inscription d'une source.....	16
Figure 3.5 — Séquence de profilage et de détection.....	16
Figure 3.6 — Séquence d'investigation d'un incident.....	17
Figure 3.7 — Séquence d'évaluation de la gouvernance IA.....	17
Figure 3.8 — Cycle de vie d'un incident DataWatch.....	18
Figure 3.9 — Classes du noyau de surveillance.....	19
Figure 3.10 — Classes d'identité et de collaboration.....	19
Figure 3.11 — Classes des moniteurs typés et révisions.....	20
Figure 3.12 — Classes du registre de gouvernance IA.....	20
Figure 3.13 — Architecture logique en couches.....	22
Figure 3.14 — Déploiement de la pile de démonstration.....	23
Figure 3.15 — Cartographie des 29 tables applicatives.....	25
Figure 3.16 — Modèle relationnel du cœur de surveillance.....	26
Figure 3.17 — Modèle relationnel identité, équipes et astreintes.....	27
Figure 3.18 — Modèle relationnel du registre de gouvernance IA.....	28

CHAPITRE 4 — RÉALISATION ET VALIDATION

Figure 4.1 — Connexion au workspace Acme Corp.....	32
Figure 4.2 — Vue Opérations et incidents prioritaires.....	33
Figure 4.3 — Catalogue des tables surveillées.....	34
Figure 4.4 — Liste filtrable des incidents.....	35
Figure 4.5 — Détail de l'incident critique orders.....	36
Figure 4.6 — Analyse IA de l'incident et actions proposées.....	37
Figure 4.7 — Profil de la table public.orders.....	38
Figure 4.8 — Recommandations de moniteurs pour public.orders.....	39
Figure 4.9 — Catalogue des moniteurs typés.....	40
Figure 4.10 — Constructeur de moniteur DSL.....	41

Figure 4.11 — Rapports hebdomadaires de santé.....	42
Figure 4.12 — Répertoire des équipes.....	43
Figure 4.13 — Membres de l'équipe Data Engineering.....	44
Figure 4.14 — Sources de données enregistrées.....	45
Figure 4.15 — Catalogue des connecteurs disponibles.....	46
Figure 4.16 — Routes d'alerte par canal et sévérité.....	47
Figure 4.17 — Préférences individuelles de notification.....	48
Figure 4.18 — File de travail de gouvernance IA.....	49
Figure 4.19 — Détail d'un système IA déclaré.....	50
Figure 4.20 — Chronologie des preuves de gouvernance.....	51
Figure 4.21 — Connexion isolée au portail staff.....	52
Figure 4.22 — Administration des organisations clientes.....	53
Figure 4.23 — Tableau de bord global du portail staff.....	54
Figure 4.24 — Détail opérationnel de l'organisation Acme Corp.....	55

VII. Liste des tableaux

CHAPITRE 1 — CADRE GÉNÉRAL

Tableau 1.1 — Acteurs et responsabilités du projet.....	3
Tableau 1.2 — Découpage des sprints et critères de sortie.....	6

CHAPITRE 2 — ÉTAT DE L'ART

Tableau 2.1 — Positionnement synthétique des approches.....	8
---	---

CHAPITRE 3 — ANALYSE ET CONCEPTION

Tableau 3.1 — Besoins fonctionnels prioritaires.....	11
Tableau 3.2 — Exigences non fonctionnelles et réponses.....	12

CHAPITRE 4 — RÉALISATION ET VALIDATION

Tableau 4.1 — Technologies principales et justification.....	31
Tableau 4.2 — Matrice de validation fonctionnelle.....	56

ANNEXES

Tableau D.1 — Structure de ai_approvals.....	62
Tableau D.2 — Structure de ai_control_evaluations.....	62
Tableau D.3 — Structure de ai_data_use_revisions.....	63
Tableau D.4 — Structure de ai_deployments.....	64
Tableau D.5 — Structure de ai_evidence.....	65
Tableau D.6 — Structure de ai_governance_incidents.....	66
Tableau D.7 — Structure de ai_release_manifests.....	66
Tableau D.8 — Structure de ai_system_versions.....	67
Tableau D.9 — Structure de ai_systems.....	67
Tableau D.10 — Structure de alert_configs.....	68
Tableau D.11 — Structure de api_keys.....	69
Tableau D.12 — Structure de check_results.....	69
Tableau D.13 — Structure de custom_monitors.....	69
Tableau D.14 — Structure de data_sources.....	70
Tableau D.15 — Structure de incidents.....	70
Tableau D.16 — Structure de invites.....	71
Tableau D.17 — Structure de monitor_evaluation_states.....	71
Tableau D.18 — Structure de monitor_revisions.....	72
Tableau D.19 — Structure de monitor_runs.....	72
Tableau D.20 — Structure de monitored_tables.....	73

Tableau D.21 — Structure de monitors.....	74
Tableau D.22 — Structure de oncall_schedules.....	75
Tableau D.23 — Structure de organizations.....	75
Tableau D.24 — Structure de staff_users.....	75
Tableau D.25 — Structure de table_profiles.....	76
Tableau D.26 — Structure de team_members.....	76
Tableau D.27 — Structure de teams.....	76
Tableau D.28 — Structure de user_notification_prefs.....	77
Tableau D.29 — Structure de users.....	77
Tableau E.1 — Effectifs du jeu de démonstration.....	79
Tableau E.2 — Organisations de démonstration.....	80
Tableau E.3 — Utilisateurs Acme sans identifiants sensibles.....	80
Tableau E.4 — Équipes Acme.....	80
Tableau E.5 — Sources Acme.....	80
Tableau E.6 — Tables surveillées chez Acme.....	81
Tableau E.7 — Incidents Acme ouverts.....	81
Tableau E.8 — Systèmes IA déclarés.....	81

VIII. Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
IA	Intelligence artificielle
LLM	Large Language Model
SaaS	Software as a Service
SLA	Service Level Agreement
JWT	JSON Web Token
ORM	Object Relational Mapping
CI	Continuous Integration

INTRODUCTION GÉNÉRALE

1.1 Contexte et motivation

La transformation numérique a placé la donnée au centre des processus de pilotage, de recommandation et d'automatisation. Le problème change d'échelle dès que quatre tensions se croisent : le volume des tables, la variété des moteurs, la vélocité des mises à jour et la véracité des valeurs. Une rupture de fraîcheur, une dérive de schéma ou une poussée silencieuse de valeurs manquantes peut alors contaminer plusieurs usages avant d'être visible. DataWatch attaque ces quatre dimensions sans aspirer les lignes métier dans sa propre base : calcul agrégé au plus près de la source, contrat commun pour les connecteurs, exécution périodique distribuée et conservation de profils horodatés.

La qualité des données ne se réduit pas à la validité syntaxique. Elle englobe notamment l'exactitude, la complétude, la cohérence, l'actualité et la traçabilité. Le modèle ISO/IEC 25012 fournit un vocabulaire utile pour exprimer ces dimensions [9]. Dans ce projet, elles sont traduites en signaux observables : volume, fraîcheur, empreinte de schéma, taux de valeurs nulles, cardinalité et distributions numériques.

1.2 Problématique et démarche

La problématique retenue est la suivante : comment concevoir une plateforme SaaS capable de détecter précocement des anomalies de qualité, de les transformer en incidents compréhensibles et de guider l'investigation, sans exposer les données entre organisations ni présenter les sorties d'un modèle génératif comme des vérités opérationnelles ? Cette question combine des enjeux de génie logiciel, d'ingénierie des données, de sécurité et d'intelligence artificielle.

La démarche suivie associe étude de l'existant, spécification des besoins, conception UML, architecture en couches, réalisation incrémentale et validation par preuves. Les décisions importantes sont reliées à des artefacts vérifiables : migrations, contrats d'API, tests, captures d'écran et jeux de démonstration. Cette discipline permet de présenter honnêtement ce qui fonctionne, ce qui reste expérimental et ce qui exigerait une validation supplémentaire avant une mise en production.

QUESTION DIRECTRICE

Comment passer d'une métrique technique isolée à une décision d'exploitation traçable, tout en maintenant l'isolation multi-tenant et une frontière explicite entre fait observé, hypothèse IA et action humaine ?

Les entreprises s'appuient de plus en plus sur des données distribuées entre applications transactionnelles, entrepôts analytiques et services cloud. Dans cet environnement, une mauvaise qualité de données ne se limite pas à une erreur technique : elle perturbe les tableaux de bord, les décisions métiers, les traitements automatisés et la confiance des utilisateurs. La difficulté consiste donc à détecter rapidement les écarts significatifs et à fournir aux opérateurs un contexte utilisable pour agir.

L'objectif de ce projet est de concevoir et réaliser DataWatch, une plateforme de surveillance de la qualité des données adaptée à un contexte SaaS multi-tenant. La solution doit inventorier les sources, suivre les tables, produire des profils, détecter des anomalies, ouvrir des incidents et acheminer des alertes. Elle doit aussi exploiter l'IA générative pour transformer des signaux techniques en explications structurées, sans confondre une recommandation avec une décision automatique.

1.3 Résultats attendus

- Une architecture multi-tenant séparant les données et les identités des organisations.
- Un profilage agrégé des tables avec métriques de volume, fraîcheur, schéma et colonnes.
- Une détection hybride : règles, z-score, Isolation Forest et STL lorsque l'historique est suffisant.
- Une gestion d'incidents, de narration IA et de routage d'alertes vérifiable localement.
- Une base de gouvernance IA observe-only qui rend les lacunes de preuve visibles sans prétendre certifier la conformité.

1.4 Organisation du rapport

Le premier chapitre présente le contexte et la démarche. Le deuxième chapitre situe la solution par rapport aux approches existantes. Le troisième chapitre analyse les besoins et expose la conception ainsi que l'architecture. Le quatrième chapitre détaille la réalisation technique et les validations. Enfin, la conclusion générale synthétise les apports et ouvre les perspectives.

CHAPITRE 1 — CADRE GÉNÉRAL ET CONDUITE DU PROJET

1.1 Introduction

Ce chapitre présente le cadre académique du projet, sa problématique et la méthode de conduite retenue. Le projet a été réalisé dans le cadre de la formation 3CI — Big Data et Intelligence Artificielle à l'ISGA Casablanca.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation de l'ISGA

L'ISGA est un établissement marocain d'enseignement supérieur qui forme des ingénieurs et des managers. Son cycle d'ingénieur s'inscrit dans un cursus de cinq années et met l'accent sur la maîtrise scientifique, la conduite de projets complexes, l'ouverture professionnelle et la capacité d'adaptation [10]. Le campus de Casablanca accueille notamment la spécialisation Intelligence Artificielle et Big Data, qui fournit le cadre académique de ce projet.

1.2.2 Environnement pédagogique du projet

Le projet a été réalisé dans le cadre du Projet de Fin d'Études de la troisième année du cycle ingénieur. L'encadrement assure la cohérence méthodologique, la validation de la problématique et le suivi des livrables. Le travail couvre le cadrage, la conception, l'implémentation, les tests, la préparation d'une démonstration et la rédaction scientifique. Il ne correspond pas à une mission réalisée pour un client externe ; l'organisme d'accueil désigne ici l'environnement académique qui porte et évalue le projet.

Tableau 1.1 — Acteurs et responsabilités du projet

Acteur	Responsabilité principale	Livrables associés
Étudiant	Analyse, conception, développement, tests et documentation	Code source, démonstration, rapport et présentation
Encadrant	Orientation scientifique et validation méthodologique	Revues, recommandations et validation du périmètre
Jury	Évaluation de la pertinence, de la réalisation et de la maîtrise	Questions, appréciation et décision académique
Utilisateur cible	Expression des besoins d'exploitation et d'investigation	Scénarios d'usage et critères d'acceptation

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Le projet est mené dans un cadre académique à l'ISGA Casablanca. L'établissement fournit l'environnement pédagogique, l'encadrement scientifique et le contexte de validation. Cette configuration oriente le travail vers une démonstration technique reproductible, documentée et utile pour une soutenance, plutôt que vers une revendication de déploiement industriel à grande échelle.

1.3 Présentation du projet

1.3.1 Contexte métier

Les équipes data exploitent des tableaux de bord, des modèles analytiques et des services alimentés par des bases hétérogènes. Lorsqu'un jeu de données devient vide, obsolète ou incohérent, l'impact se propage aux décisions métiers. Les contrôles artisanaux détectent parfois l'écart, mais ils fournissent rarement une chronologie, un niveau de sévérité, un propriétaire et une procédure de résolution. DataWatch vise à réunir ces éléments dans un même parcours.

1.3.2 Périmètre fonctionnel

Le périmètre couvre l'authentification par organisation, le registre de sources hétérogènes, la découverte des schémas, la sélection des tables, le profilage périodique et la gestion complète des incidents. Le cœur Big Data se trouve dans ce passage de sources variées à des profils compacts, calculés par agrégats et traités hors requête web par une file de tâches. Le cœur IA combine règles déterministes, z-score, Isolation Forest et décomposition STL ; la narration générative intervient ensuite, sur un contexte borné. Les moniteurs typés et le petit prototype de gouvernance IA en mode observation complètent cette chaîne. La facturation réelle, les garanties de disponibilité et l'exploitation à grande échelle restent hors du périmètre de preuve du PFE.

1.3.3 Utilisateurs cibles

La solution cible d'abord le data engineer chargé de fiabiliser les flux, l'analytics engineer responsable des modèles et indicateurs, et le responsable data qui souhaite une vision synthétique du risque. L'administrateur de l'organisation configure les accès et les routes d'alerte. Une séparation complémentaire réserve le portail staff à l'administration de la plateforme, afin que la gestion commerciale et la gestion des données clientes ne partagent pas la même surface d'accès.

1.3.4 Problématique

Trois difficultés structurent le besoin. Premièrement, les systèmes sources utilisent des dialectes et des capacités différentes ; il faut donc normaliser les profils sans masquer les limites de chaque connecteur. Deuxièmement, une alerte utile doit éviter la répétition :

plusieurs échecs associés à une table doivent enrichir le même incident tant qu'il reste ouvert. Troisièmement, l'IA générative doit être utile à l'enquête sans devenir l'autorité qui décide de l'existence ou de la gravité de l'incident.

Les équipes data disposent rarement d'une vue centralisée de la santé de leurs actifs. Les contrôles sont souvent isolés dans des scripts, les alertes manquent de contexte et les anomalies statistiques demandent une expertise difficile à mobiliser immédiatement. Dans une plateforme SaaS, la difficulté augmente : l'isolement entre organisations, le chiffrement des connexions et la traçabilité des actions deviennent des exigences de conception.

1.3.5 Solution proposée

La réponse retenue est une architecture asynchrone dans laquelle la mesure précède toujours l'interprétation. Un profileur exécute une requête agrégée sur la source, les détecteurs évaluent les métriques persistées, puis le service d'incident applique les règles de sévérité et de déduplication. Ce n'est qu'après cette étape que la narration IA reçoit un contexte borné. Cette séquence préserve une preuve technique indépendante du fournisseur LLM.

PRINCIPE DE CONCEPTION

Les règles et détecteurs créent l'incident ; le LLM l'explique. Une narration indisponible ou invalide ne doit jamais effacer le signal technique ni empêcher l'opérateur d'accéder aux mesures.

DataWatch centralise les sources de données, les tables surveillées, les profils et les incidents. Le système exécute des vérifications périodiques, agrège les résultats et déduplique les incidents ouverts par table. Les alertes sont routées selon le canal et la sévérité. Une narration LLM, validée selon un schéma structuré, complète la fiche incident par des hypothèses et des recommandations de diagnostic.

1.4 Méthodologie de travail

1.4.1 Méthode Agile

La méthode adoptée reprend les principes de Scrum sans reproduire artificiellement tous les rôles d'une équipe complète. Le backlog est organisé par verticales démontrables, les tâches sont limitées à un résultat observable et chaque fin de sprint comprend une revue des critères d'acceptation. Les anomalies découvertes durant les tests réintègrent le backlog. La documentation et les preuves ne sont pas reportées à la fin : elles font partie de la définition de terminé.

La conduite du projet s'appuie sur une démarche Agile courte et itérative. Les besoins sont priorisés, implémentés dans une verticale complète, puis vérifiés par des tests et une démonstration avant d'ouvrir l'incrément suivant.

Une démarche Agile et itérative a été adoptée. Chaque incrément vise une verticale complète : besoin, modèle de données, API, interface, tests et documentation. Cette approche limite les fonctionnalités décoratives et rend chaque avancée démontrable sur une pile locale reproductible.

1.4.2 Organisation en sprints

Tableau 1.2 — Découpage des sprints et critères de sortie

Sprint	Objectif	Critère de sortie
S1	Cadrage et environnement	Périmètre, risques et pile locale documentés
S2	Identités et multi-tenancy	Authentification et isolation testées
S3	Sources et profilage	Connexion, découverte et profil persistant
S4	Détection et incidents	Anomalie seedée et incident déduplicué
S5	Narration et alertes	Résumé structuré et message livré
S6	Moniteurs et gouvernance	Révisions traçables et contrôles observe-only
S7	Stabilisation PFE	Tests, captures, démonstration et rapport

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Le projet a été découpé en incréments fonctionnels : fondations multi-tenant, profilage et détection, narration et alertes, moniteurs et gouvernance, puis préparation de la démonstration. Chaque sprint se termine par une vérification du parcours couvert.

1.4.3 Diagramme de Gantt et jalons

Le calendrier ne simule pas une succession parfaitement linéaire. L'architecture démarre pendant le cadrage ; la détection chevauche le profilage ; les tests commencent avant la fermeture fonctionnelle. La ligne rouge du 31 août borne la période évaluée. Au-delà, la barre grise assume le statut réel du projet : le produit continue.

La figure 1.1 replace les travaux dans les trois mois retenus pour le PFE, du 1er juin au 31 août, puis distingue la continuité du produit après cette échéance académique.

Planification du PFE

Du 1er juin au 31 août 2026 — continuité produit matérialisée après la remise

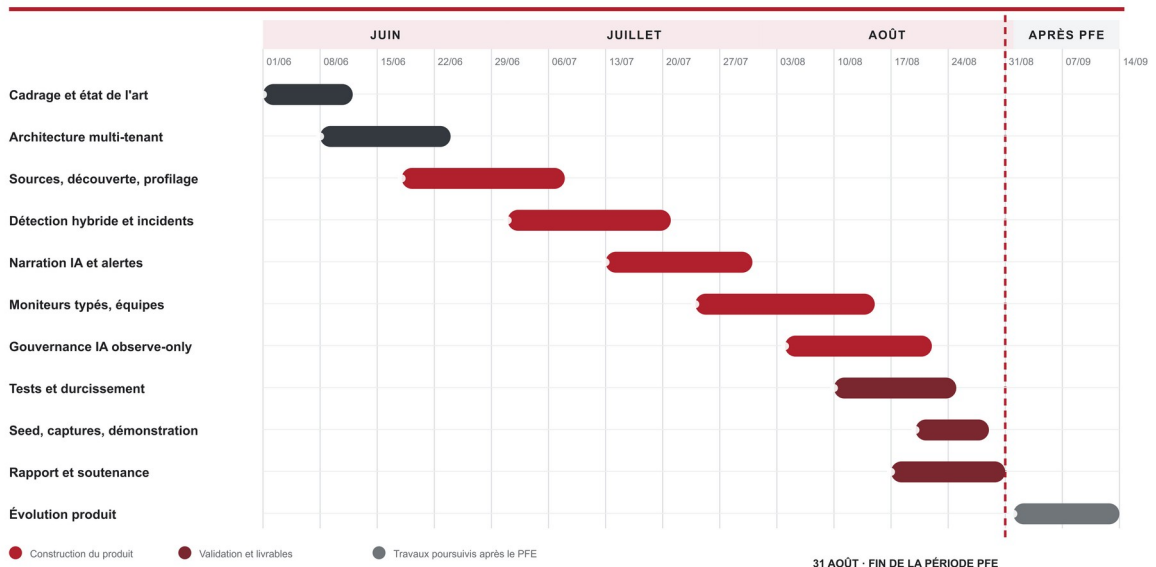


Figure 1.1 — Planification du PFE du 1er juin au 31 août et continuité du produit

La lecture horizontale révèle des chevauchements assumés : le profilage commence avant la clôture de l'architecture, tandis que les tests accompagnent les derniers incréments. Le jalon du 31 août ferme l'évaluation du PFE, pas le développement de DataWatch.

Le planning associe chaque incrément à un objectif vérifiable et à un livrable exploitable dans la démonstration. La figure 1.1 synthétise les sept sprints, leurs chevauchements et leurs principaux jalons.

Le projet est organisé en sept sprints. Les deux premiers cadrent les besoins et les fondations multi-tenant. Les troisième et quatrième couvrent le profilage, la détection et la gestion d'incidents. Les cinquième et sixième stabilisent les alertes, les moniteurs et la gouvernance IA. Le dernier sprint est consacré aux tests, à la démonstration et à la rédaction.

Le diagramme de Gantt de la figure 1.1 matérialise les chevauchements nécessaires entre conception, implémentation et validation. Chaque sprint se termine par une preuve visible : modèle migré, API testée, parcours navigateur ou jeu de démonstration reproductible.

1.5 Conclusion

Le cadrage met en évidence une double exigence : rendre la qualité des données observable et conserver une discipline de preuve adaptée à une solution SaaS. Le chapitre suivant étudie les approches qui inspirent la solution et précise son originalité.

CHAPITRE 2 — ÉTAT DE L'ART ET POSITIONNEMENT

2.1 Introduction

La surveillance de qualité des données recouvre plusieurs familles de mécanismes : validation déclarative, contrôle statistique, apprentissage automatique et observabilité opérationnelle. Aucune de ces familles ne répond seule à l'ensemble des besoins ; leur combinaison doit rester explicable et gouvernable.

2.2 Approches existantes

2.2.1 Contrôles déclaratifs

Les frameworks déclaratifs formalisent les attentes sous forme de règles lisibles et versionnables. Great Expectations associe des Expectations à des lots de données et produit des résultats de validation [11]. SodaCL décrit des métriques et des seuils dans un langage YAML [12]. Cette famille est adaptée aux contrats connus, mais exige que l'équipe explicite les règles et maintienne les seuils lorsque le comportement des données évolue.

2.2.2 Détection statistique et apprentissage non supervisé

Les seuils statiques sont insuffisants lorsque le volume suit une tendance ou une saisonnalité. Le z-score compare une observation à une moyenne et un écart-type historiques ; Isolation Forest repère des observations rares dans un espace multivarié ; STL sépare tendance, saison et résidu. Ces méthodes réduisent l'effort de configuration, mais nécessitent un historique minimal et peuvent produire des faux positifs lorsque le contexte métier change.

2.2.3 Observabilité et gestion d'incidents

Une plateforme d'observabilité complète ne se limite pas à exécuter des tests. Elle relie un signal à un actif, conserve l'historique, attribue une sévérité, notifie les responsables et facilite l'investigation. La valeur opérationnelle dépend donc autant de la déduplication, du routage et de la traçabilité que du détecteur lui-même. Cette observation justifie l'orientation de DataWatch vers une verticale allant de la mesure à l'action.

Tableau 2.1 — Positionnement synthétique des approches

Approche	Point fort	Limite principale	Apport retenu
Règles déclaratives	Lisibles et auditables	Configuration manuelle	Moniteurs typés et versionnés
Seuils statistiques	Adaptation à l'historique	Sensibles aux ruptures de régime	Z-score et évolution temporelle

Approche	Point fort	Limite principale	Apport retenu
Apprentissage non supervisé	Détection multivariée	Explication plus difficile	Isolation Forest avec score visible
Observabilité SaaS	Parcours incident intégré	Coût et dépendance fournisseur	Incidents, alertes et vues d'enquête
Scripts internes	Adaptation métier rapide	Maintenance et audit faibles	Extension SQL bornée et contrôlée

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Les solutions existantes se répartissent généralement entre contrôles déclaratifs, observabilité automatisée et scripts spécifiques. Les contrôles déclaratifs sont faciles à versionner mais nécessitent une définition manuelle. Les plateformes d'observabilité accélèrent la détection, au prix d'un coût et d'une dépendance au fournisseur. Les scripts internes s'adaptent au contexte, mais deviennent vite difficiles à maintenir et à auditer.

DataWatch combine ces apports dans un prototype pédagogique : règles explicites, détection statistique, apprentissage non supervisé, gestion d'incidents et narration structurée. L'originalité ne réside pas dans un algorithme isolé, mais dans l'enchaînement vérifiable du signal jusqu'à l'action humaine, avec une séparation nette entre faits mesurés, hypothèses générées et décisions de l'opérateur.

2.3 Originalité de la solution

Le positionnement n'est pas celui d'un remplacement complet des outils industriels. DataWatch assemble plutôt, dans un prototype SaaS cohérent, des briques rarement montrées ensemble dans un PFE : diversité des sources, profilage agrégé, exécution asynchrone, détection hybride et traitement opérationnel de l'incident. Le volume n'est pas déplacé inutilement ; il est condensé en métriques sur la source. La variété est rendue explicite par un registre de capacités. La vitesse est absorbée par la planification et les workers. La véracité, enfin, devient une suite de preuves datées au lieu d'un simple voyant rouge. La narration IA reste subordonnée à ces faits mesurés.

La seconde originalité concerne la gouvernance. Les preuves, versions et évaluations sont séparées des états mutables de déploiement. Les contrôles peuvent conclure pass, fail, unknown, unsupported, not applicable ou error ; un manque d'information n'est donc pas transformé artificiellement en conformité. Le mode observe-only rend le risque visible sans bloquer une publication et sans revendiquer une certification juridique.

Le DSL de moniteurs complète cette chaîne par des définitions strictes, versionnées et liées au schéma observé. Une révision est validée puis prévisualisée avant activation ; l'exécution utilise un plan déterministe et un état d'évaluation séparé de l'historique immuable. Ce choix évite qu'une modification silencieuse change le comportement d'un contrôle déjà audité.

La contribution de DataWatch réside dans la chaîne intégrée allant du profilage à l'incident, puis à une narration IA structurée et à l'alerte. Le système ne présente pas le LLM comme un détecteur autonome : les contrôles déterministes et les métriques calculées restent la source de l'incident. L'IA reformule le contexte disponible et propose des actions, ce qui conserve une frontière claire entre observation, recommandation et décision humaine.

2.4 Conclusion

L'étude montre que l'explicabilité et le contrôle des limites sont aussi importants que le choix d'un algorithme. Le chapitre suivant traduit ces constats en besoins, modèles et composants d'architecture.

CHAPITRE 3 — ANALYSE, CONCEPTION ET ARCHITECTURE

3.1 Introduction

La conception de DataWatch s'appuie sur la séparation des responsabilités : interface de pilotage, API métier, exécution asynchrone, persistance, cache et connecteurs. Les décisions de conception privilégient l'isolation tenant, la traçabilité et la possibilité de valider chaque étape.

3.2 Analyse du projet

3.2.1 Objectifs de la solution

L'objectif général est décliné en résultats vérifiables : enregistrer une source sans stocker ses secrets en clair ; réduire une table potentiellement volumineuse à un profil agrégé et horodaté ; distribuer les travaux de surveillance sans bloquer l'API ; choisir un détecteur selon l'historique disponible ; expliquer chaque contrôle par une valeur observée, un score et une plage attendue ; conserver un incident unique tant que le défaut persiste ; livrer l'alerte selon sa sévérité ; puis produire une narration structurée qui sépare résumé, causes plausibles et actions recommandées.

L'analyse vise à construire une plateforme qui centralise les actifs de données, détecte les écarts de qualité, crée des incidents contextualisés et accompagne l'investigation sans automatiser la décision humaine. Les objectifs suivants structurent les besoins fonctionnels et non fonctionnels.

3.2.2 Besoins fonctionnels

Tableau 3.1 — Besoins fonctionnels prioritaires

ID	Besoin	Critère d'acceptation
BF-01	Authentifier un utilisateur dans son organisation	Le contexte tenant est établi avant tout accès métier
BF-02	Enregistrer et tester une source	Le statut de connexion et l'erreur éventuelle sont visibles
BF-03	Découvrir et surveiller une table	L'actif est planifié avec intervalle et sensibilité
BF-04	Produire un profil	Volume, fraîcheur, schéma et métriques colonnes sont persistés
BF-05	Détecter une anomalie	Chaque résultat contient état, observation et justification

ID	Besoin	Critère d'acceptation
BF-06	Gérer un incident	Création, acquittement, affectation et résolution sont traçables
BF-07	Alerter les responsables	Le canal respecte la sévérité minimale configurée
BF-08	Assister l'investigation	La narration structurée reste distincte des faits mesurés

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Les besoins fonctionnels couvrent six capacités cohérentes : connecter une source et vérifier sa disponibilité ; sélectionner les tables à surveiller ; produire des profils périodiques ; appliquer des règles et détecteurs hybrides ; ouvrir, affecter, acquitter puis résoudre les incidents ; enfin, acheminer les alertes et documenter les preuves de gouvernance IA.

À ces fonctions s'ajoutent des contraintes transversales : isolation multi-tenant, chiffrement des secrets, traçabilité des révisions, idempotence des exécutions, explication des signaux et dégradation explicite lorsqu'une dépendance externe est indisponible. Ces exigences structurent directement l'architecture présentée dans les figures suivantes.

3.2.3 Besoins non fonctionnels

Tableau 3.2 — Exigences non fonctionnelles et réponses de conception

Exigence	Risque traité	Réponse retenue
Sécurité	Exposition de secrets ou données inter-tenant	Chiffrement par organisation et filtres tenant
Fiabilité	Doublons et reprises incohérentes	Idempotence, états explicites et déduplication
Performance	Lecture coûteuse des tables sources	Agrégats SQL sur la source, cache borné et exécution Celery
Maintenabilité	Régression lors d'une évolution	Contrats typés, migrations et tests automatisés
Traçabilité	Perte du contexte d'une décision	Révisions immuables et résultats horodatés
Utilisabilité	Surcharge cognitive pendant un incident	Hiérarchie visuelle et parcours guidé

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

- Sécurité : chiffrement des configurations de connexion et séparation des organisations.
- Fiabilité : tâches idempotentes, déduplication des incidents et reprise planifiée.
- Performance : profilage par requêtes agrégées plutôt que lecture ligne par ligne.
- Maintenabilité : API typée, migrations versionnées et tests de régression.
- Traçabilité : profils, résultats de contrôles, incidents et versions de moniteurs conservés.
- Explicabilité : scores, plages attendues et raisons de contrôle visibles dans l'interface.

3.3 Conception de la solution

3.3.1 Acteurs et cas d'utilisation

Le diagramme de cas d'utilisation distingue le membre d'une organisation, son administrateur et l'opérateur staff. Le membre consulte les actifs et traite les incidents ; l'administrateur ajoute les sources, configure les tables, les moniteurs et les alertes ; le staff gère le cycle de vie des organisations depuis un portail séparé. Les services externes - bases, fournisseur LLM et canaux de notification - sont représentés comme systèmes secondaires.

La figure 3.1 montre les relations include entre le profilage, l'exécution des contrôles et la création d'un incident, ainsi que les extensions conditionnelles liées à la narration et à l'alerte. Cette représentation évite de confondre une action utilisateur avec une tâche planifiée. Elle met également en évidence que l'acquittement et la résolution demeurent des décisions humaines.

La figure 3.1 condense le périmètre fonctionnel autour de trois acteurs : l'administrateur d'organisation, l'opérateur data et le personnel DataWatch.

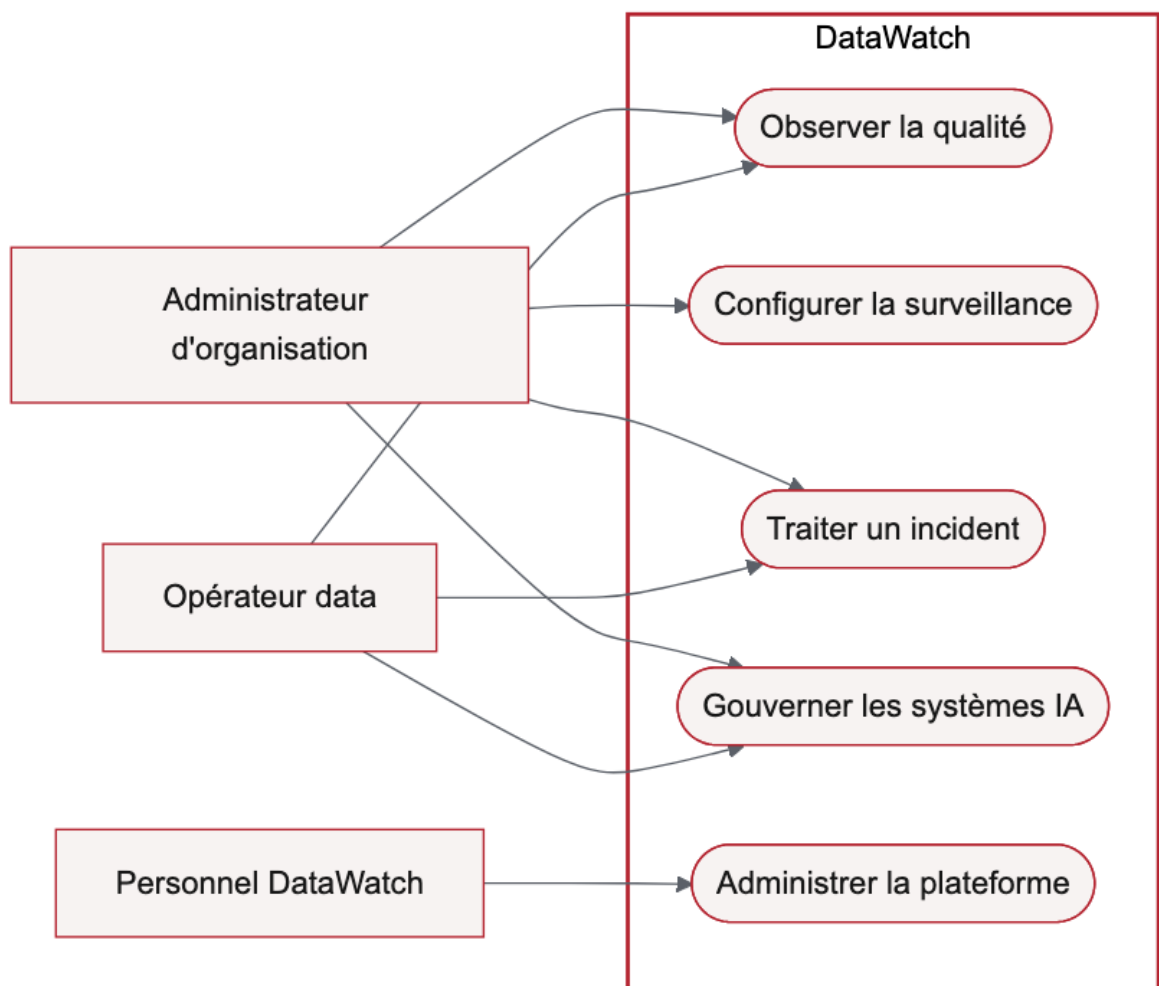


Figure 3.1 — Vue globale des acteurs et domaines fonctionnels

Cette séparation évite de confondre exploitation d'un workspace et administration de la plateforme. Le personnel DataWatch administre le SaaS ; il ne traite pas les incidents à la place du client.

La figure 3.2 détaille les actions réalisées dans un workspace client, depuis la configuration initiale jusqu'à l'investigation d'un incident.

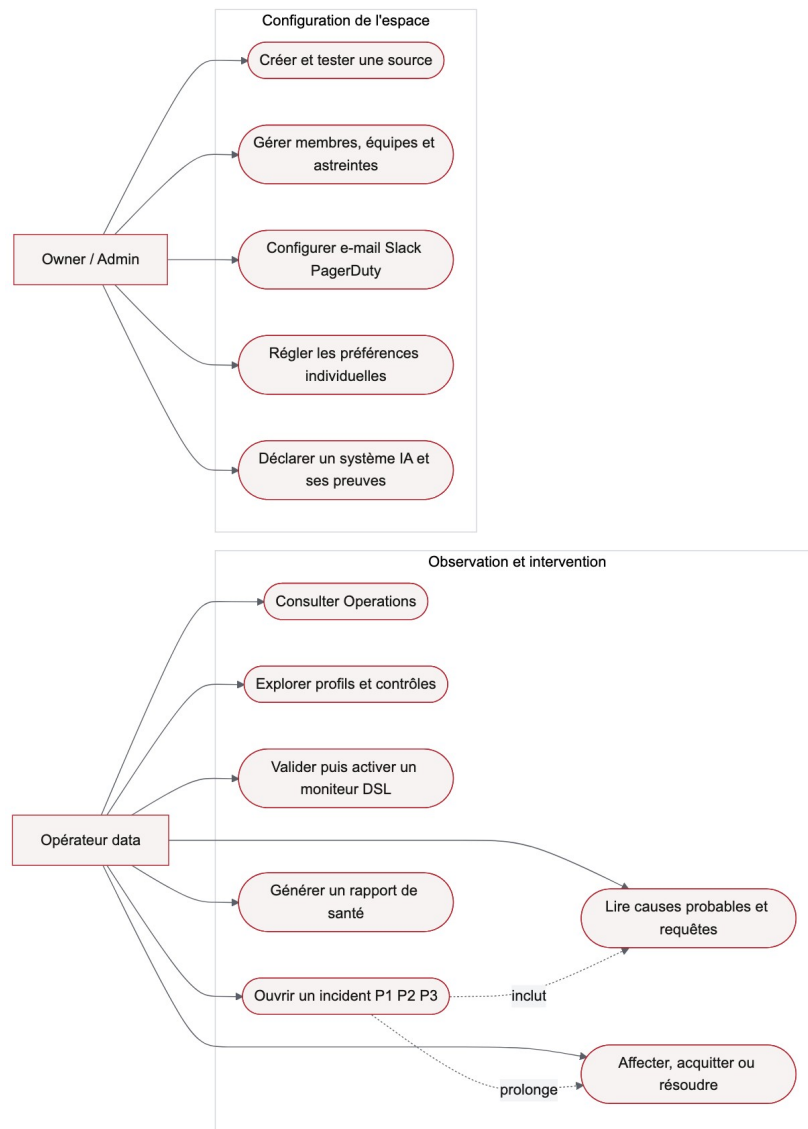


Figure 3.2 — Cas d'utilisation du workspace client

Deux responsabilités ressortent. Le propriétaire configure sources, membres et notifications ; l'opérateur consulte les profils, crée des moniteurs et pilote le cycle de vie des incidents. Les cas communs restent soumis à l'isolation du tenant.

La figure 3.3 isole les cas d'utilisation réservés à l'administration interne de DataWatch.

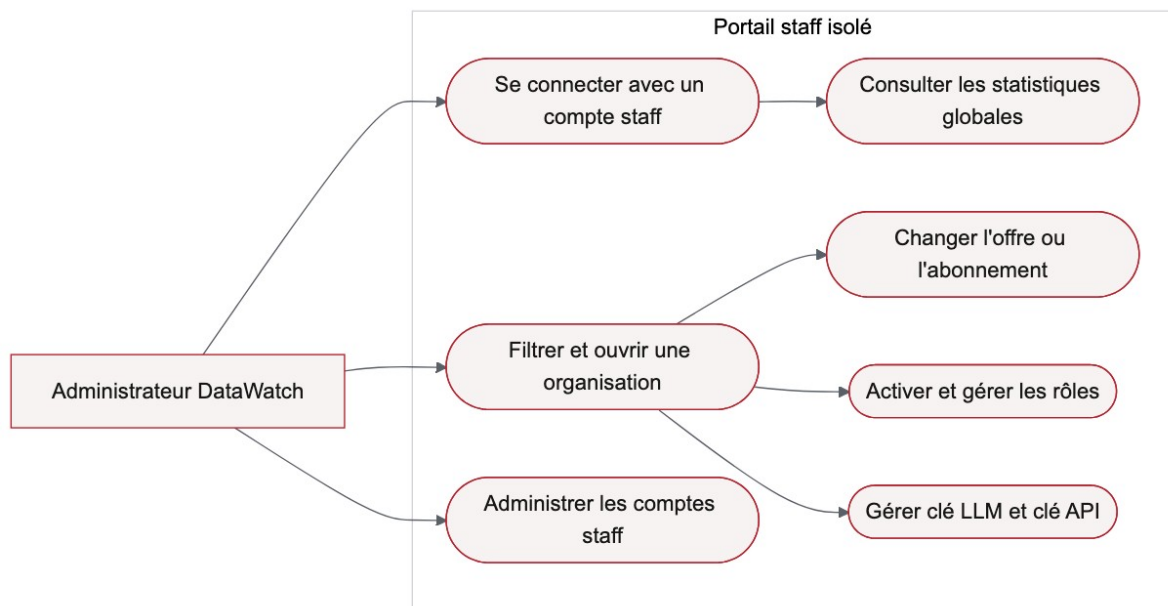


Figure 3.3 — Cas d'utilisation du portail staff

Le compte staff consulte les indicateurs globaux, ouvre une organisation, gère son offre et administre les comptes internes. Ce périmètre possède son propre mécanisme d'authentification et ne réutilise pas une session client.

Le cas d'utilisation central commence lorsqu'un utilisateur configure une source et sélectionne une table. Le système programme un profilage, calcule les métriques, exécute les contrôles, crée ou met à jour un incident, prépare une narration et achemine l'alerte. L'utilisateur conserve la maîtrise : il examine l'incident, l'assigne, l'acquitte ou le résout selon son enquête.

3.3.2 Diagramme de séquence

Le scénario nominal débute par le déclenchement d'APScheduler. L'API place l'identifiant de la table dans la file Redis ; le worker recharge ensuite la configuration, dérive la clé de déchiffrement de l'organisation et instancie le connecteur. Le profil est persistant avant l'exécution des détecteurs. Si au moins un contrôle échoue, le service d'incident crée ou enrichit l'incident, puis enchaîne narration et notification.

Deux variantes sont importantes. Si la source est indisponible, l'erreur est enregistrée sans fabriquer de métrique. Si le fournisseur LLM échoue ou renvoie un format invalide, l'incident technique reste consultable et l'échec de narration est explicite. Ce comportement de dégradation garantit que la couche d'assistance ne devient pas un point unique de défaillance.

La figure 3.4 suit l'inscription d'une source, de la saisie du formulaire jusqu'à l'activation de la surveillance d'une table découverte.

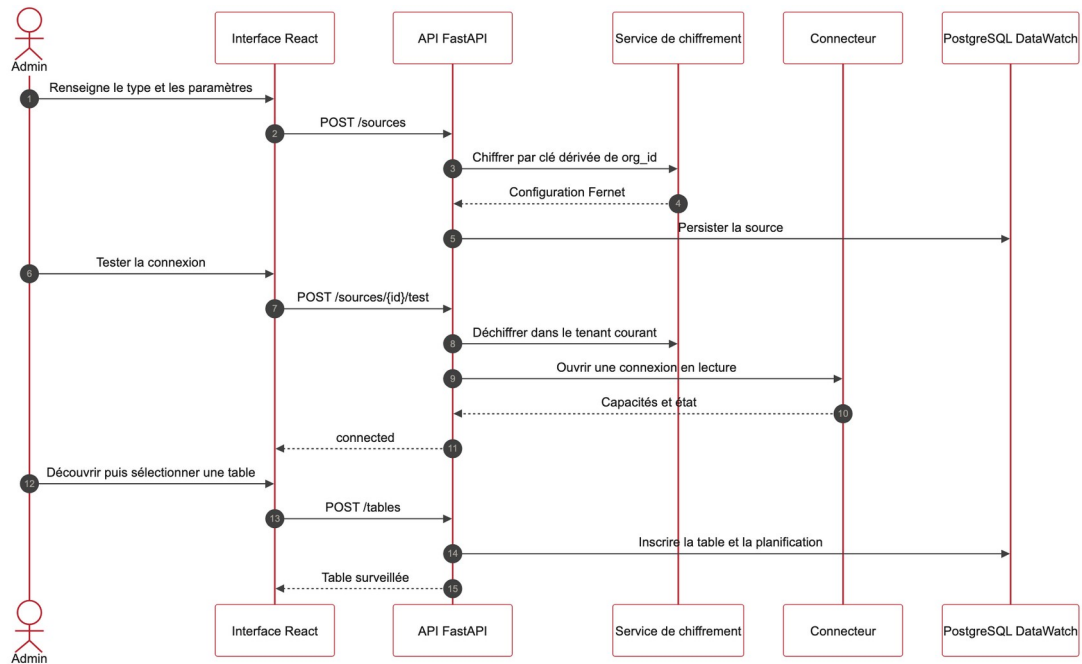


Figure 3.4 — Séquence de connexion et d'inscription d'une source

Le secret est chiffré avec une clé dérivée de l'organisation avant persistance. La connexion est ensuite testée par l'adaptateur concerné ; seules ses capacités déclarées et les tables accessibles sont renvoyées à l'interface.

La figure 3.5 expose la chaîne asynchrone qui transforme une planification en profil, résultats de contrôle et incident éventuel.

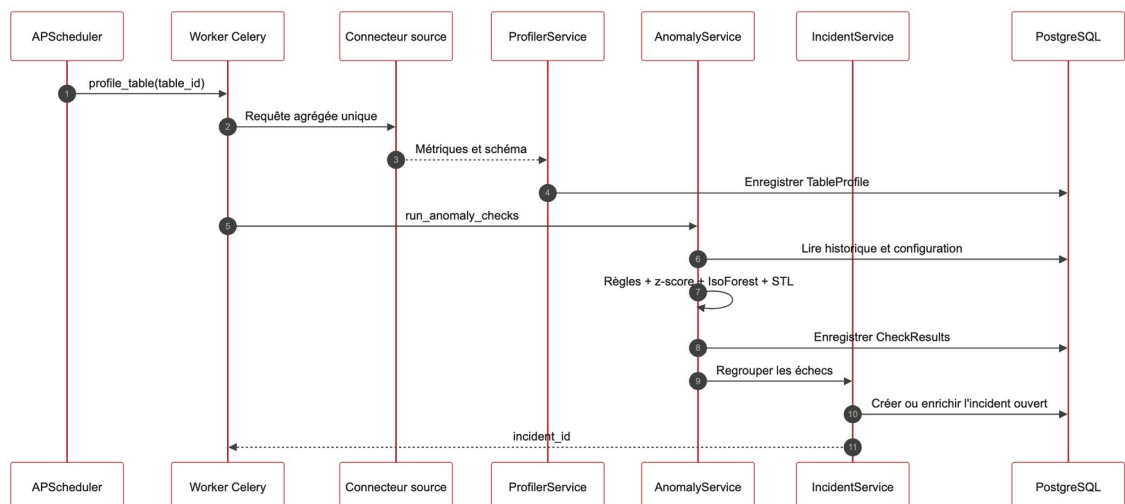


Figure 3.5 — Séquence de profilage et de détection

APScheduler déclenche le worker Celery. Une requête agrégée produit le profil, puis les règles, le z-score, Isolation Forest et STL sont évalués selon l'historique disponible. L'incident n'est créé ou enrichi qu'après persistance des résultats.

La figure 3.6 décrit l'enquête menée depuis l'ouverture d'un incident critique dans l'interface.

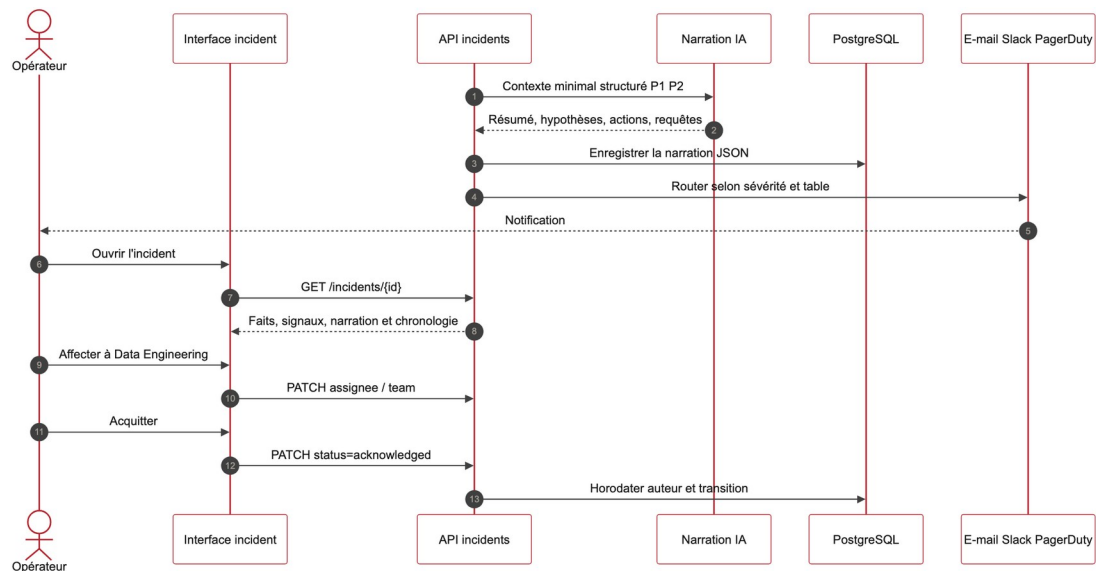


Figure 3.6 — Séquence d'investigation d'un incident

L'API réunit mesures, signaux, chronologie et narration structurée. L'opérateur peut ensuite affecter l'incident, l'acquitter ou le résoudre ; chaque transition est horodatée, tandis que l'alerte externe reste un canal de diffusion et non la source de vérité.

La figure 3.7 montre comment une version déclarée et un manifeste actif sont rapprochés des preuves disponibles avant l'évaluation des contrôles de gouvernance IA.

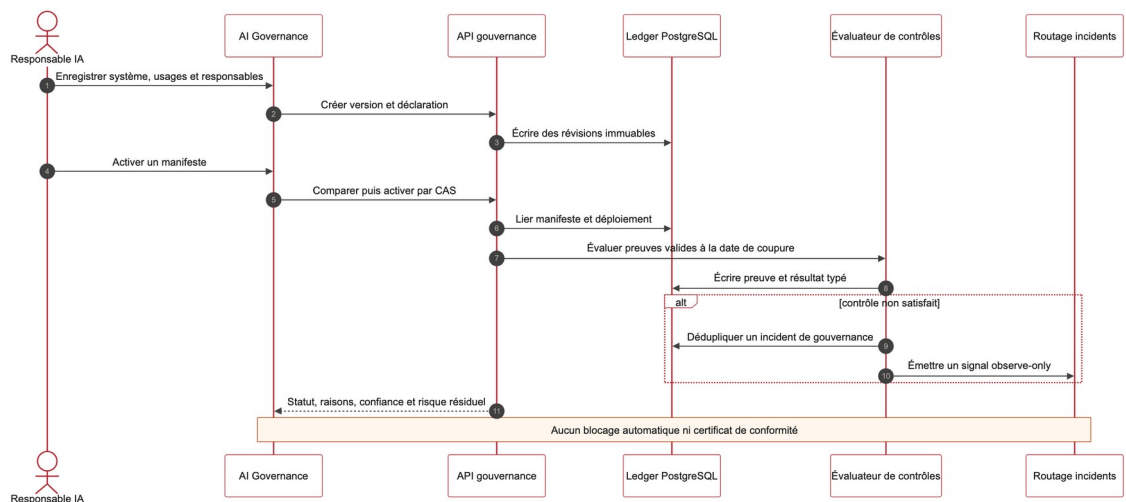


Figure 3.7 — Séquence d'évaluation de la gouvernance IA

Le registre écrit des résultats immuables et peut ouvrir un incident de gouvernance. La dernière branche est volontairement non bloquante : le prototype émet un signal observe-only, sans autoriser ni interdire une mise en production.

La figure 3.8 représente les états successifs d'un incident et les décisions accessibles à l'opérateur.

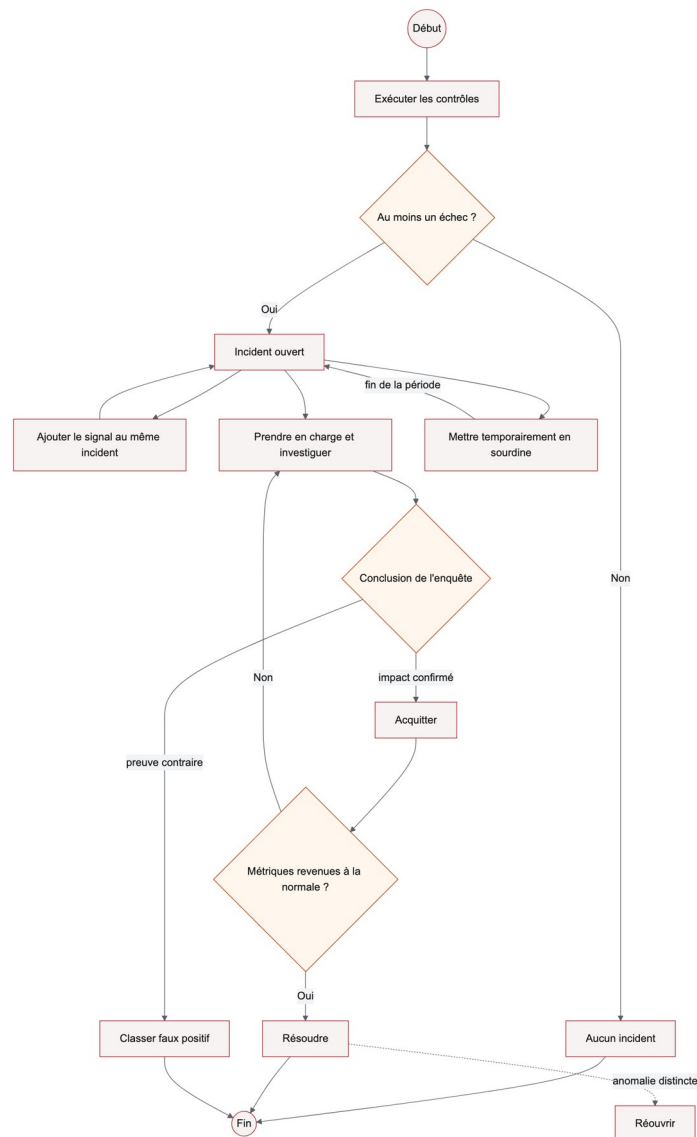


Figure 3.8 — Cycle de vie d'un incident DataWatch

Un signal peut être regroupé avec un incident déjà ouvert. Après acquittement, l'investigation mène soit à une résolution, soit à la déclaration d'un faux positif ; une nouvelle anomalie peut rouvrir le travail au lieu d'effacer l'historique.

La figure 3.2 détaille la succession des échanges depuis le déclenchement planifié jusqu'à l'acquittement humain. Elle montre que le profil, les résultats des contrôles et l'incident sont persistés avant la narration et l'alerte.

3.3.3 Diagramme de classes

Le modèle de domaine s'organise autour de l'agrégat Organization. DataSource et MonitoredTable décrivent les actifs ; TableProfile et CheckResult constituent la preuve de mesure ; Incident regroupe les occurrences liées ; AlertConfig exprime le routage. Le sous-

domaine des moniteurs sépare Monitor, identité stable, de MonitorRevision, définition append-only, et de MonitorRun, trace d'exécution. Cette séparation évite qu'une modification réécrive l'historique.

Quatre vues remplacent un diagramme monolithique devenu illisible sur A4. Elles conservent les cardinalités et les frontières de domaine : surveillance, collaboration, moniteurs versionnés, puis gouvernance IA.

La figure 3.9 extrait les classes qui portent la surveillance, du connecteur enregistré jusqu'au résultat de contrôle.

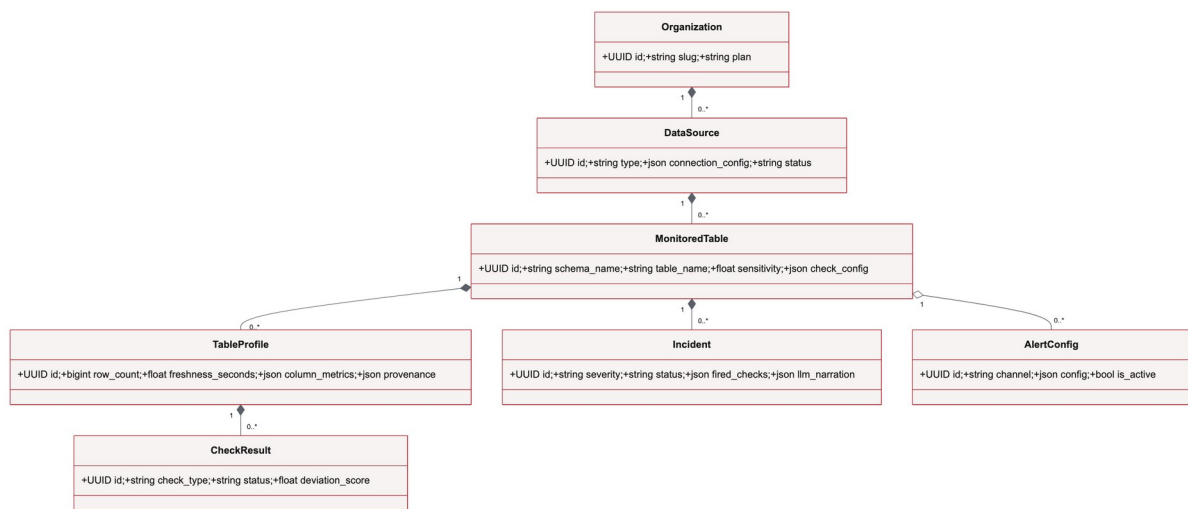


Figure 3.9 — Classes du noyau de surveillance

DataSource possède les actifs surveillés ; chaque MonitoredTable accumule des TableProfile et peut porter un Incident ainsi qu'une configuration d'alerte. CheckResult conserve le fait mesuré séparément de la décision d'incident.

La figure 3.10 regroupe identité, invitations, équipes, préférences de notification, astreintes et clés d'API autour de l'organisation.

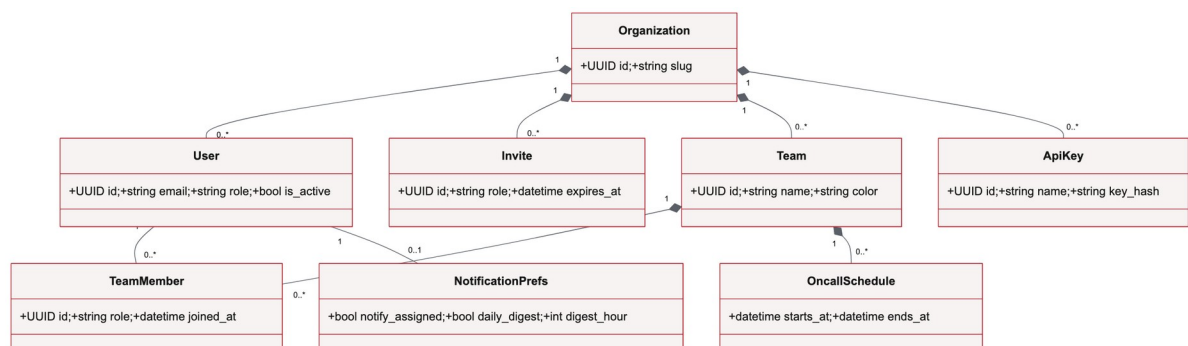


Figure 3.10 — Classes d'identité et de collaboration

Organization constitue la frontière de tenant. Les associations TeamMember et OncallSchedule rendent l'affectation explicite, alors que StaffUser reste hors de ce graphe afin de préserver la séparation entre comptes clients et comptes internes.

La figure 3.11 détaille le modèle des moniteurs typés et la conservation de leurs révisions.

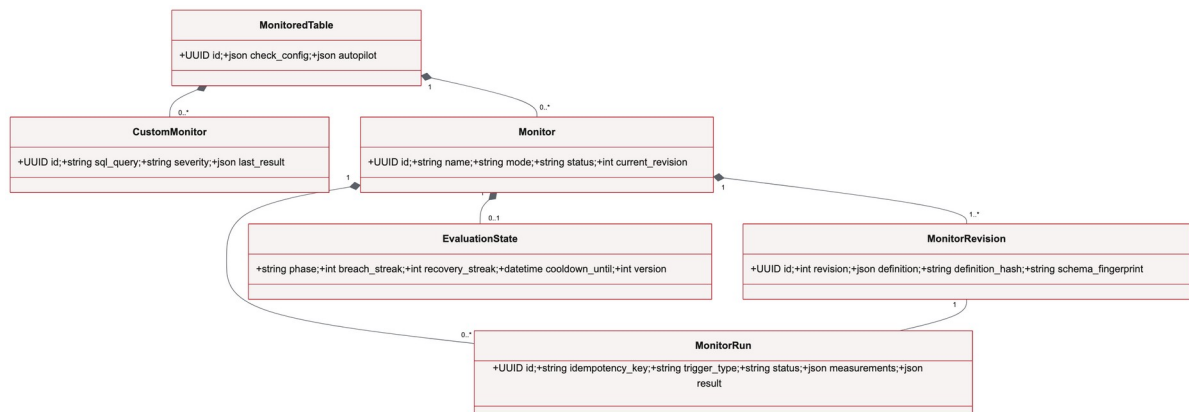


Figure 3.11 — Classes des moniteurs typés et révisions

Monitor porte l'identité stable et pointe vers une révision courante. MonitorRevision, MonitorRun et les états d'évaluation séparent définition, exécution et temporisation des alertes ; une modification ne réécrit donc pas la règle qui a produit un résultat passé.

La figure 3.12 présente les classes du registre expérimental de gouvernance IA.

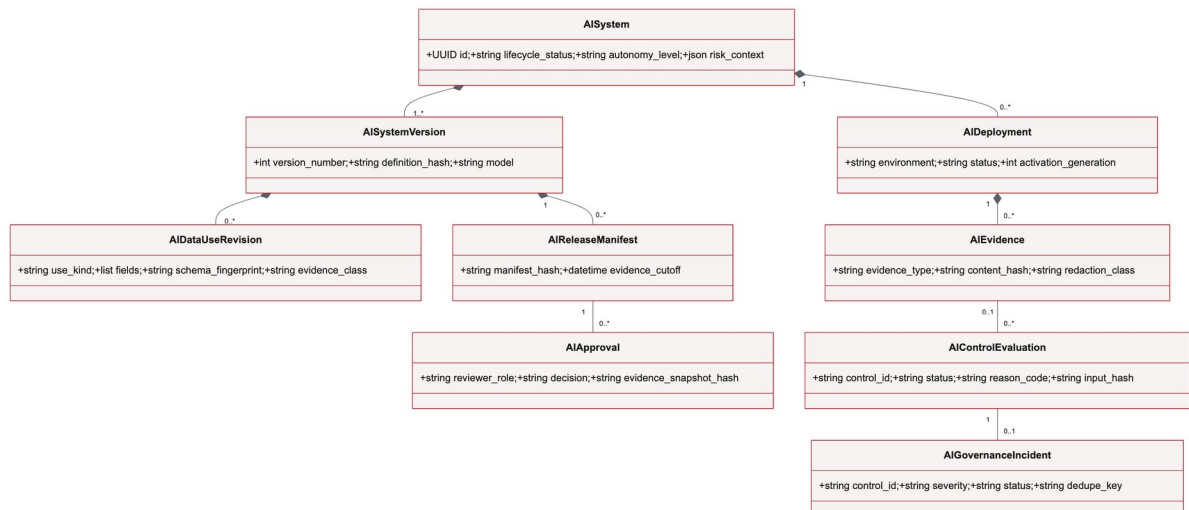


Figure 3.12 — Classes du registre de gouvernance IA

AISystem sert de racine à des versions, usages déclarés, manifestes, déploiements, approbations et preuves. Les évaluations et incidents conservent leur provenance, mais ce modèle de traçabilité reste observe-only et ne représente pas une certification automatisée.

Le modèle de la figure 3.3 se concentre sur le noyau métier. L'organisation possède ses utilisateurs et ses sources ; chaque source expose des tables surveillées, dont les profils alimentent les contrôles, incidents, moniteurs et alertes. Les révisions de moniteur sont immuables afin de conserver l'historique des définitions.

3.4 Architecture proposée

3.4.1 Vue logique en couches

La couche présentation regroupe l'application React et ses composants d'état. Elle ne contient pas les règles d'isolation ; chaque appel est contrôlé côté serveur. La couche API expose des contrats REST, valide les entrées et orchestre les services. La couche métier applique profilage, détection, incidents, narration et politiques de plan. La couche d'infrastructure fournit persistance, broker, cache et connecteurs.

3.4.2 Exécution asynchrone

Le profilage peut dépasser la durée acceptable d'une requête interactive. Il est donc exécuté par Celery. Redis porte la file et certains caches, tandis qu'APScheduler maintient un job par table active. Cette chaîne forme le plan d'exécution distribué du produit : l'API accepte la demande, la file découple le rythme des sources de celui des workers, puis chaque tâche persiste un résultat rejouable. Au redémarrage, les actifs en retard peuvent être remis en file. Les services restent asynchrones et les tâches synchrones utilisent des enveloppes contrôlées, ce qui évite de maintenir deux implémentations métier.

3.4.3 Architecture des connecteurs

Un contrat BaseConnector uniformise le test de connexion, la découverte, l'introspection et l'exécution de la requête de profil. Les capacités sont déclarées par connecteur plutôt que supposées. PostgreSQL constitue la verticale stable ; DuckDB et SQLite sont positionnés en bêta ; les autres adaptateurs demandent des validations réelles supplémentaires. Ce modèle permet d'étendre le catalogue sans présenter tous les connecteurs comme équivalents.

La figure 3.13 situe les composants de DataWatch entre interface, API, traitements différés, persistance et services externes.

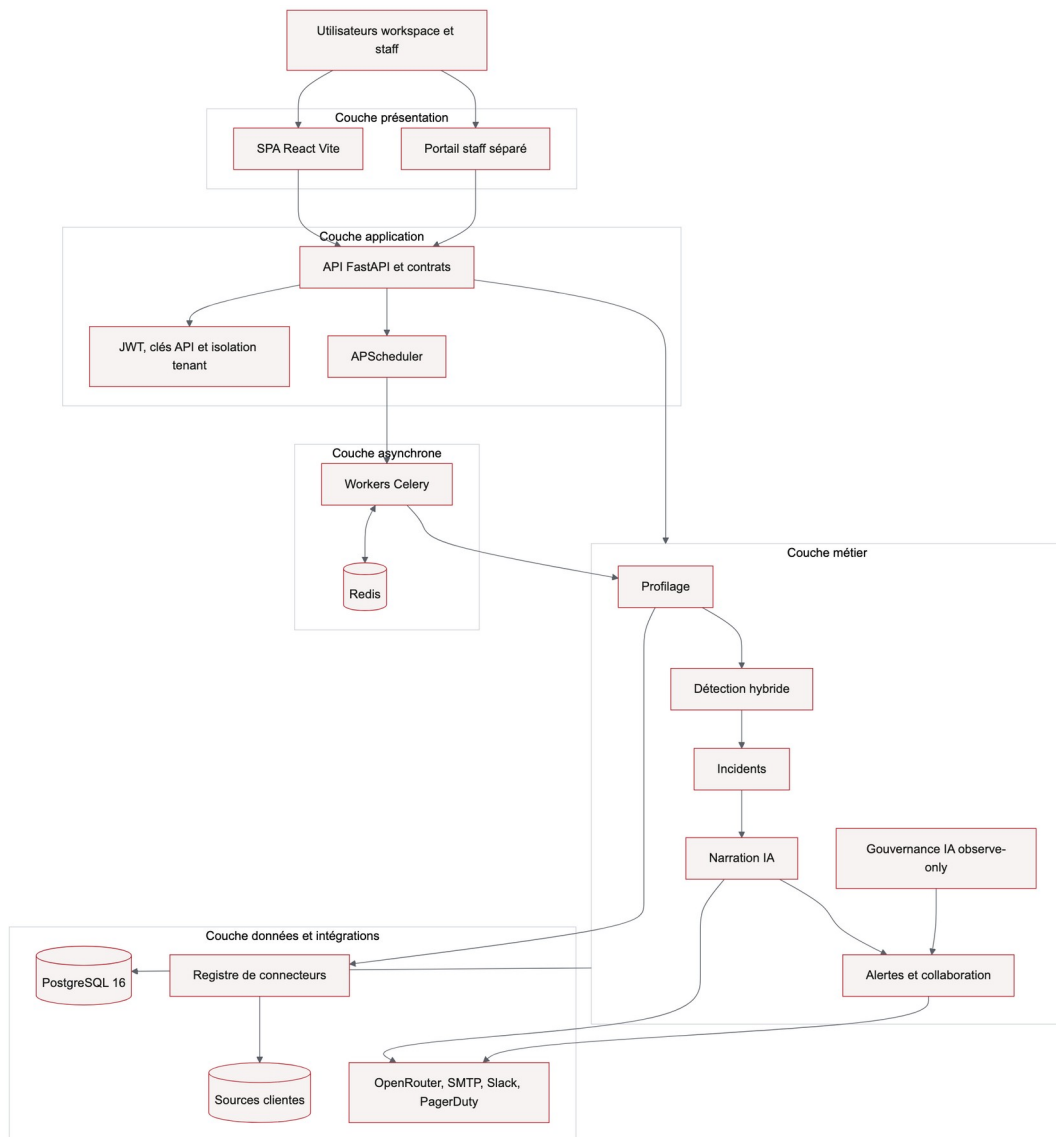


Figure 3.13 — Architecture logique en couches

L'API FastAPI centralise contrats et autorisations ; Celery absorbe les opérations longues ; PostgreSQL conserve l'historique et Redis transporte ou met en cache les travaux temporaires. Les connecteurs demeurent derrière une couche d'adaptation commune.

La figure 3.14 traduit cette architecture dans la machine Docker Compose utilisée pour la démonstration.

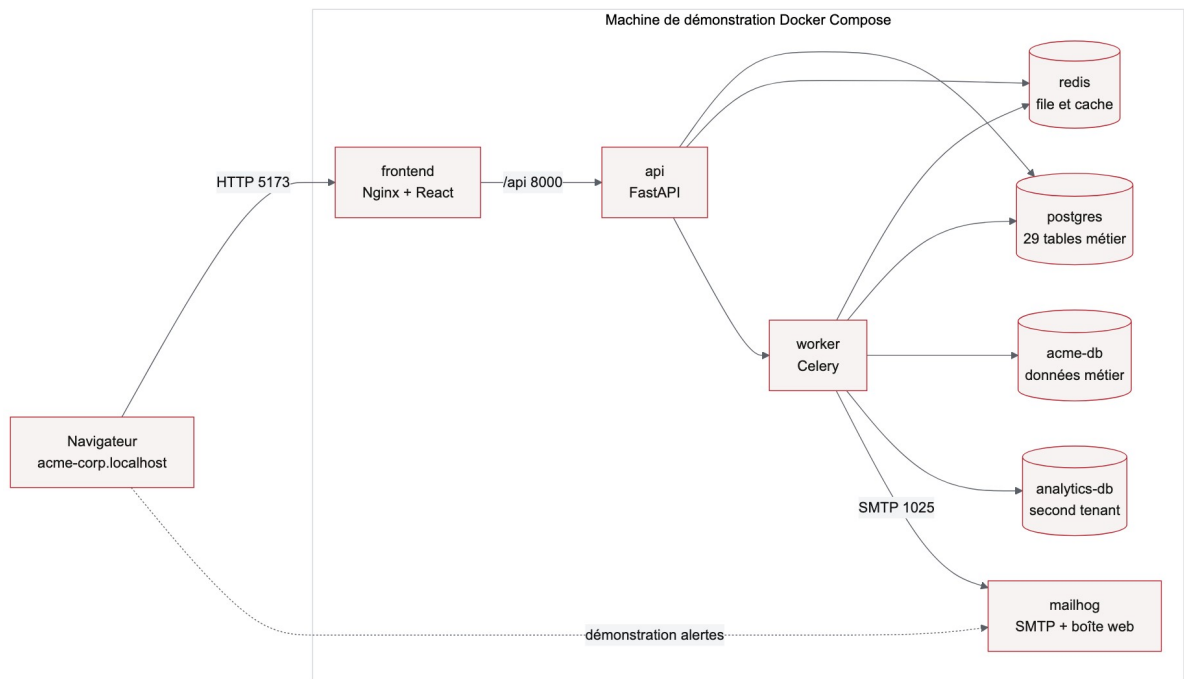


Figure 3.14 — Déploiement de la pile de démonstration

Le navigateur atteint le frontend Nginx puis l'API. Le worker partage PostgreSQL et Redis avec l'API, interroge la base métier Acme et remet les courriels à MailHog ; cette topologie prouve le parcours local, pas une haute disponibilité de production.

Le cycle de profilage démarre dans APScheduler, qui place une tâche dans Redis. Celery déchiffre la configuration de la source dans le contexte de l'organisation, construit une requête agrégée, puis réduit la table à un profil compact : volume, fraîcheur, schéma, nullité, cardinalité et distributions. Une fois ce snapshot persisté, les règles, le z-score, Isolation Forest, STL et les moniteurs personnalisés peuvent l'évaluer selon l'historique disponible. La narration et l'alerte viennent après l'incident ; une dépendance externe défaillante ne supprime donc jamais le fait technique observé.

L'architecture est organisée en couches. La couche présentation est une application React. La couche application expose une API FastAPI et les règles métier. La couche d'exécution asynchrone regroupe Celery et le planificateur. PostgreSQL persiste les entités métier et Redis sert de broker et de cache. Les connecteurs encapsulent les dialectes de sources afin de préserver un contrat de profilage commun.

3.5 Modèle de données et sécurité

3.5.1 Isolation multi-tenant

Chaque entité métier est rattachée directement ou indirectement à une organisation. Les requêtes protégées résolvent le tenant à partir du JWT ou de la clé API, puis filtrent les données selon ce contexte. Des clés étrangères composites renforcent cette preuve au niveau relationnel pour les sous-domaines sensibles. Le portail staff utilise une identité distincte afin d'éviter la confusion entre administration de plateforme et opérations clientes.

3.5.2 Protection des secrets

Les mots de passe sont hachés et les clés API ne sont montrées qu'à leur création. Les configurations de connexion sont chiffrées avec Fernet ; la clé effective est dérivée du secret maître et de l'identifiant de l'organisation au moyen de HKDF. Ainsi, un chiffré copié d'un tenant vers un autre ne peut pas être déchiffré dans le nouveau contexte. Les journaux et captures évitent les secrets bruts.

3.5.3 Intégrité et conservation

Les profils et résultats sont horodatés afin de reconstruire l'état ayant mené à un incident. Les révisions de moniteur et les preuves de gouvernance sont immuables lorsque leur valeur d'audit l'exige. Les suppressions ordinaires ne doivent pas effacer silencieusement un registre de preuve. Cette rigueur a un coût en stockage et nécessite, pour une industrialisation, une politique d'export et de rétention gouvernée.

3.5.4 Structure relationnelle complète

Le schéma physique compte 29 tables applicatives. La vue d'ensemble révèle quatre masses nettes : identité et collaboration, surveillance, moniteurs typés, gouvernance IA. Trois vues relationnelles détaillées suivent. Le dictionnaire exhaustif — colonnes, types, nullabilité, clés et effectifs seedés — est reporté en annexe afin de garder ce chapitre respirable.

La figure 3.15 offre une carte compacte des 29 tables applicatives avant leur décomposition en vues lisibles.

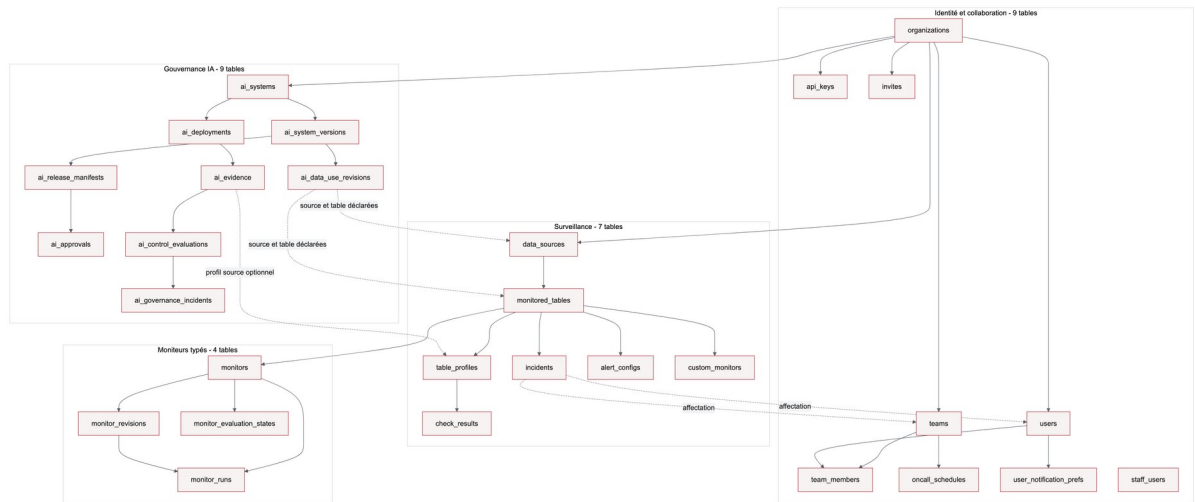


Figure 3.15 — Cartographie des 29 tables applicatives

Quatre zones apparaissent : identité et collaboration, surveillance, moniteurs versionnés et gouvernance IA. Les relations convergent vers `organizations`, clé de l'isolation multi-tenant, tandis que `staff_users` demeure volontairement séparée.

La figure 3.16 agrandit le modèle relationnel utilisé par le parcours principal de qualité des données.

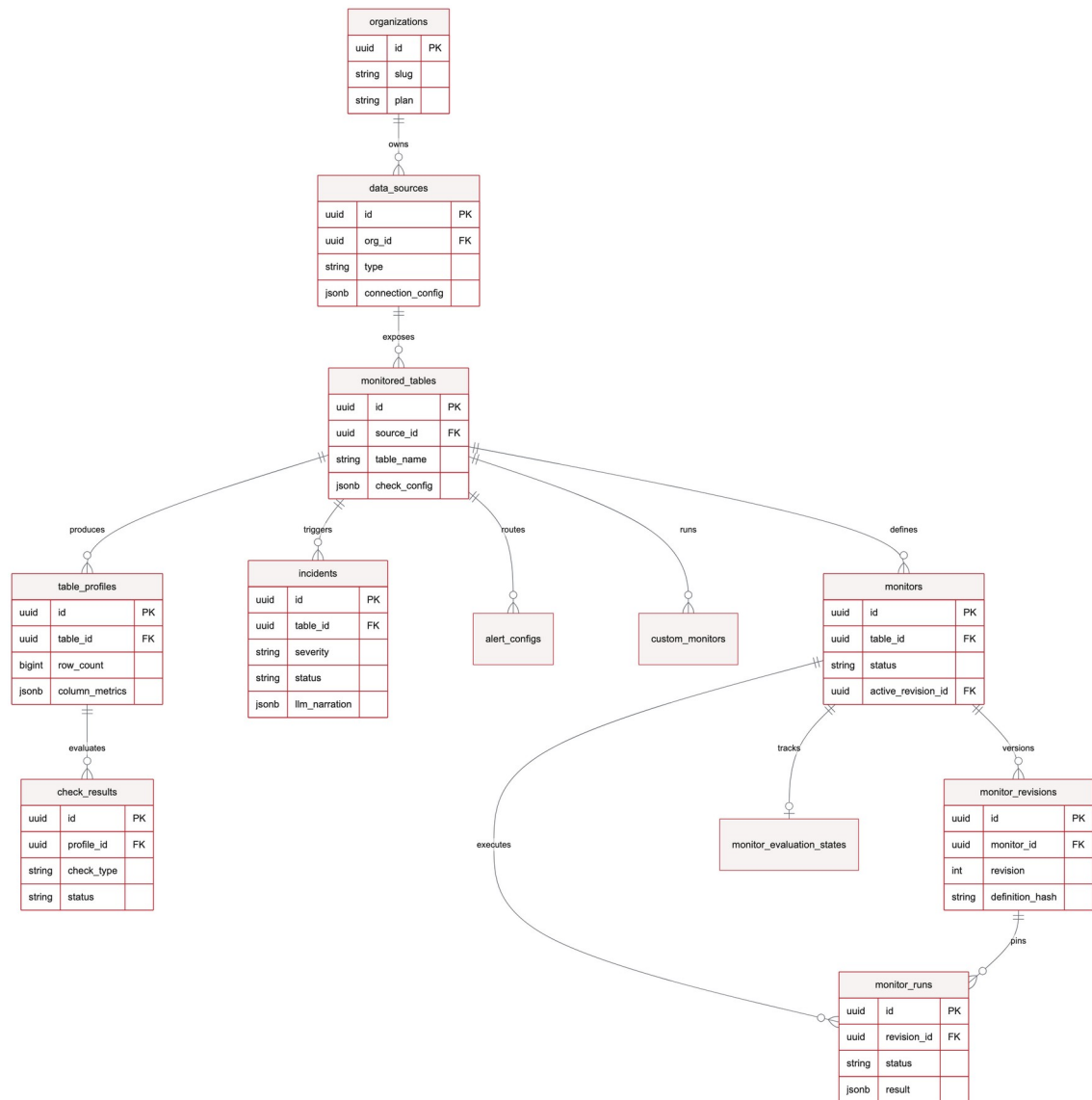


Figure 3.16 — Modèle relationnel du cœur de surveillance

Les clés étrangères relient source, table, profils, résultats et incidents sans dupliquer le fait observé. Les configurations d’alerte et moniteurs restent rattachés au même tenant, ce qui permet de vérifier l’appartenance lors de chaque opération.

La figure 3.17 isole les tables d'identité, d'invitation, d'équipe et d'astreinte.

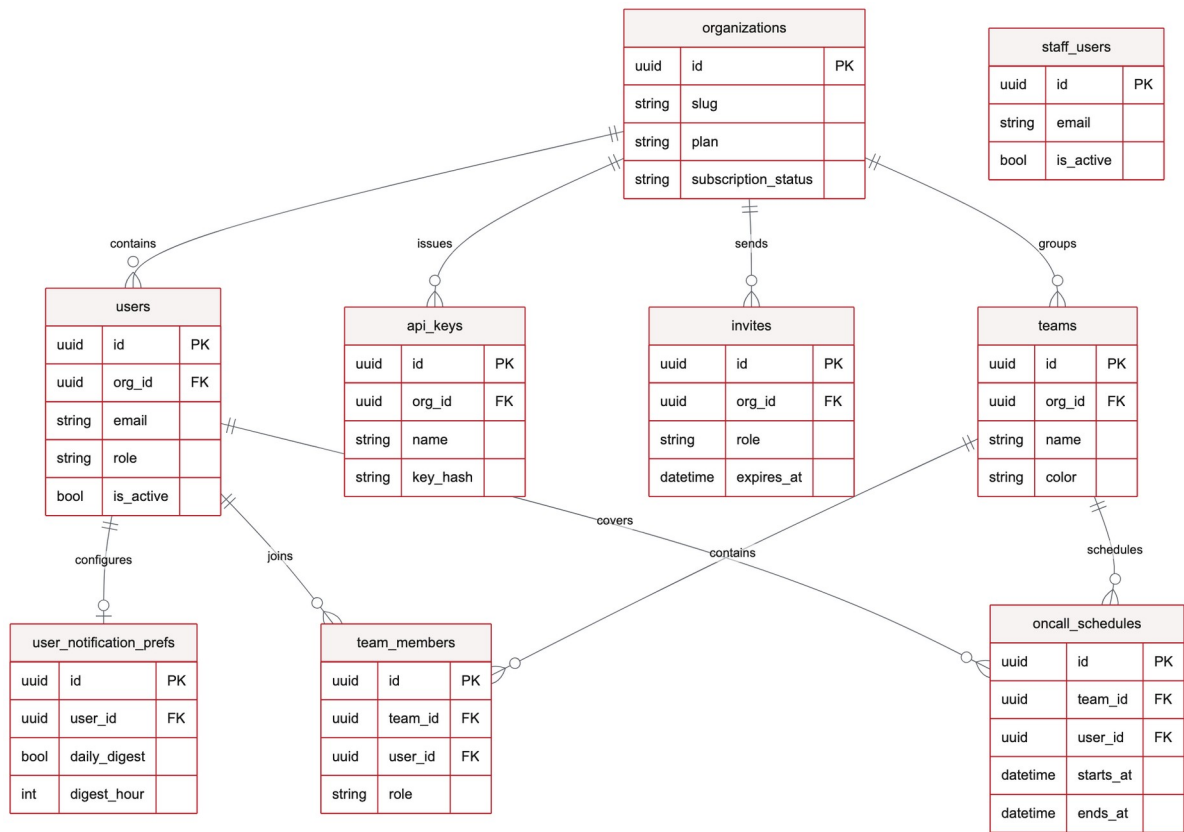


Figure 3.17 — Modèle relationnel identité, équipes et astreintes

Les rôles d'organisation et d'équipe sont distincts. Cette granularité autorise, par exemple, un membre ordinaire du workspace à devenir responsable opérationnel d'une équipe sans lui donner les droits d'administration du tenant.

La figure 3.18 développe la partie relationnelle consacrée au prototype de gouvernance IA.



Figure 3.18 — Modèle relationnel du registre de gouvernance IA

Les révisions et manifestes sont conservés comme instantanés, puis reliés aux déploiements, preuves et évaluations. Cette structure prépare la traçabilité et le jeu ; elle ne démontre ni l'équité du modèle ni son usage juridique conforme.

La sécurité repose également sur des clés étrangères composites qui prouvent l'appartenance des entités à une même organisation. Les révisions, résultats terminaux et preuves de gouvernance sont append-only lorsque leur valeur d'audit doit être conservée. Les clés API sont stockées sous forme de hachage et les secrets de connexion sont chiffrés avec une clé dérivée par organisation.

Les tables organizations, users, data_sources, monitored_tables, table_profiles, check_results, incidents et alert_configs portent la verticale de surveillance. Les modèles de moniteurs sont versionnés afin qu'une modification ne réécrive pas silencieusement une

définition active. Les secrets de connexion sont stockés chiffrés au niveau applicatif avec des clés Fernet dérivées par organisation. Cette séparation renforce l'isolation inter-tenant et réduit l'exposition des configurations.

3.6 Gouvernance de la couche IA

Le plan de contrôle distingue l'identité du système IA, ses versions, les usages de données déclarés, les manifestes de release, les déploiements, les approbations, les preuves et les évaluations. Les manifestes sont adressés par leur contenu et l'activation utilise un mécanisme compare-and-swap. Les évaluations référencent précisément la preuve et la version utilisées, ce qui permet de reproduire une décision sans considérer l'état courant comme historique.

Le périmètre actuel observe une verticale PostgreSQL/pgvector pour les chaînes RAG. Les requêtes sont bornées, exécutées en lecture seule et limitées à des agrégats et métadonnées. Aucune donnée client brute n'est envoyée au registre de preuve. Les états inconnu, non pris en charge ou périmé restent visibles. Ce choix réduit le risque de greenwashing technique et constitue un point de discussion important pour le jury.

La gouvernance IA est conçue en mode observe-only. Elle conserve des descripteurs de preuve metadata-only, des versions, des empreintes de contenu et des résultats de contrôles immuables. Les états inconnus, obsolètes ou non pris en charge restent visibles comme lacunes de preuve. Cette conception facilite l'audit technique mais ne constitue ni une certification juridique, ni une preuve de conformité, ni un mécanisme de blocage en production.

3.7 Conclusion

La conception relie les exigences à une architecture composable et testable. Le chapitre suivant présente les choix de réalisation, les interfaces et les résultats de validation.

CHAPITRE 4 — IMPLÉMENTATION TECHNIQUE ET TESTS

4.1 Introduction

Cette phase concrétise les choix de conception dans une pile conteneurisée. L'objectif n'est pas de revendiquer une capacité de production universelle, mais de démontrer une verticale complète, reproductible et vérifiable localement.

4.2 Workflow proposé

4.2.1 Enregistrement et planification

Après validation de la connexion, l'utilisateur choisit une table, un intervalle, une sensibilité et éventuellement une colonne de fraîcheur. La création enregistre l'actif puis programme le contrôle. Un bootstrap d'autopilot peut proposer des contrôles de base à partir du schéma ; les recommandations considérées risquées ou reposant sur du SQL personnalisé sont laissées en attente de revue.

4.2.2 Profilage et détection

Le profileur construit une requête unique regroupant comptage, fraîcheur, cardinalité et statistiques de colonnes. Le résultat devient un snapshot `TableProfile`. Les règles déterministes s'appliquent immédiatement ; les méthodes historiques s'activent seulement lorsque le nombre de profils est suffisant. Le score et la plage attendue sont persistés afin que l'interface puisse expliquer pourquoi un contrôle a échoué.

4.2.3 Incident, narration et alerte

Les échecs sont regroupés par table. Un incident déjà ouvert reçoit la nouvelle occurrence au lieu d'être dupliqué. La sévérité P1 est réservée aux ruptures les plus critiques, notamment table vide ou fraîcheur dépassée ; les dérives de schéma et cumuls d'échecs produisent généralement P2 ; les écarts isolés restent P3. La narration utilise ensuite un contexte compact, validé par un schéma, et les alertes filtrent les routes selon leur seuil minimal.

Lorsqu'une table est enregistrée, le planificateur crée un job périodique. Le worker profile la table par agrégats SQL, persiste un snapshot, puis exécute les contrôles. En cas d'échec, le service d'incident déduplique ou enrichit l'incident ouvert. Une nouvelle occurrence déclenche la narration IA, validée par un schéma, puis l'envoi d'alertes vers les canaux configurés.

4.3 Technologies utilisées

Tableau 4.1 — Technologies principales et justification

Couche	Technologie	Rôle et justification
Interface	React, Vite, Tailwind	SPA réactive, composants réutilisables et construction rapide
API	Python 3.12, FastAPI	Contrats typés, validation Pydantic et I/O asynchrones
Persistence	PostgreSQL 16, SQLAlchemy	Transactions, contraintes, JSONB et migrations
Traitements	Celery, APScheduler	Tâches différées et planification par actif
Broker/cache	Redis	File de tâches, modèles et résultats temporaires
IA	API compatible OpenAI	Narration structurée avec fournisseur configurable
Exploitation	Docker Compose	Pile locale reproductible et seed déterministe
Validation	Pytest, Playwright	Tests de services et parcours navigateur

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Aucune technologie n'est ici décorative. PostgreSQL conserve l'historique et les contraintes ; Redis et Celery encaissent la cadence variable des profils ; les connecteurs absorbent la diversité des dialectes ; scikit-learn et statsmodels portent les détecteurs non supervisés et temporels ; l'API compatible OpenAI transforme enfin les faits déjà calculés en explication structurée. L'architecture Big Data et IA vient de cet enchaînement, pas de l'ajout nominal d'un outil isolé.

Le choix des technologies suit les points de pression réels du produit : FastAPI expose des contrats asynchrones ; PostgreSQL verrouille transactions, historique et isolation ; Redis et Celery découplent la cadence de collecte du temps de réponse utilisateur ; les bibliothèques statistiques exécutent les détecteurs ; React rend l'enquête lisible ; Docker Compose fige la démonstration. L'ensemble peut évoluer composant par composant sans casser la chaîne de preuve de bout en bout.

La réalisation repose sur une pile volontairement cohérente. Python 3.12 et FastAPI portent l'API asynchrone et les contrats REST. SQLAlchemy 2.0 et PostgreSQL 16 assurent la persistance transactionnelle, les contraintes d'intégrité et l'historique JSONB. Celery et Redis exécutent les tâches de profilage hors requête utilisateur, tandis qu'APScheduler orchestre les contrôles périodiques.

Côté interface, React, Vite et Tailwind structurent les écrans de pilotage et d'investigation. Docker Compose rend l'environnement reproductible pour la démonstration. Pytest vérifie les services et Playwright rejoue les parcours visibles. Ce choix d'outils privilégie

l'observabilité, la séparation des responsabilités et la possibilité de tester chaque couche indépendamment.

4.4 Présentation des interfaces

La conception visuelle suit une logique de divulgation progressive. L'écran Opérations répond d'abord à trois questions : que se passe-t-il, quel actif est touché et quelle action est prioritaire ? Le détail expose ensuite les signaux, la chronologie et la narration. Les paramètres techniques restent accessibles, mais ne concurrencent pas les éléments nécessaires à la décision immédiate.

La page Opérations est le point d'entrée de la démonstration. Elle met en évidence la file d'incidents, les actifs surveillés et l'état des sources. Le détail d'incident privilégie les signaux clés, la narration IA et les actions d'investigation. Les identifiants techniques et les vérifications secondaires restent accessibles sans surcharger la lecture initiale.

4.4.1 Vue Opérations

Le parcours commence avant le tableau de bord. Le sous-domaine dans l'URL fixe le workspace ; la page de connexion reste volontairement sobre. Une fois authentifié, l'opérateur reçoit la file priorisée et le catalogue des actifs, pas un mur de graphiques décoratifs.

La figure 4.1 ouvre le parcours réel par l'écran de connexion du workspace Acme Corp.

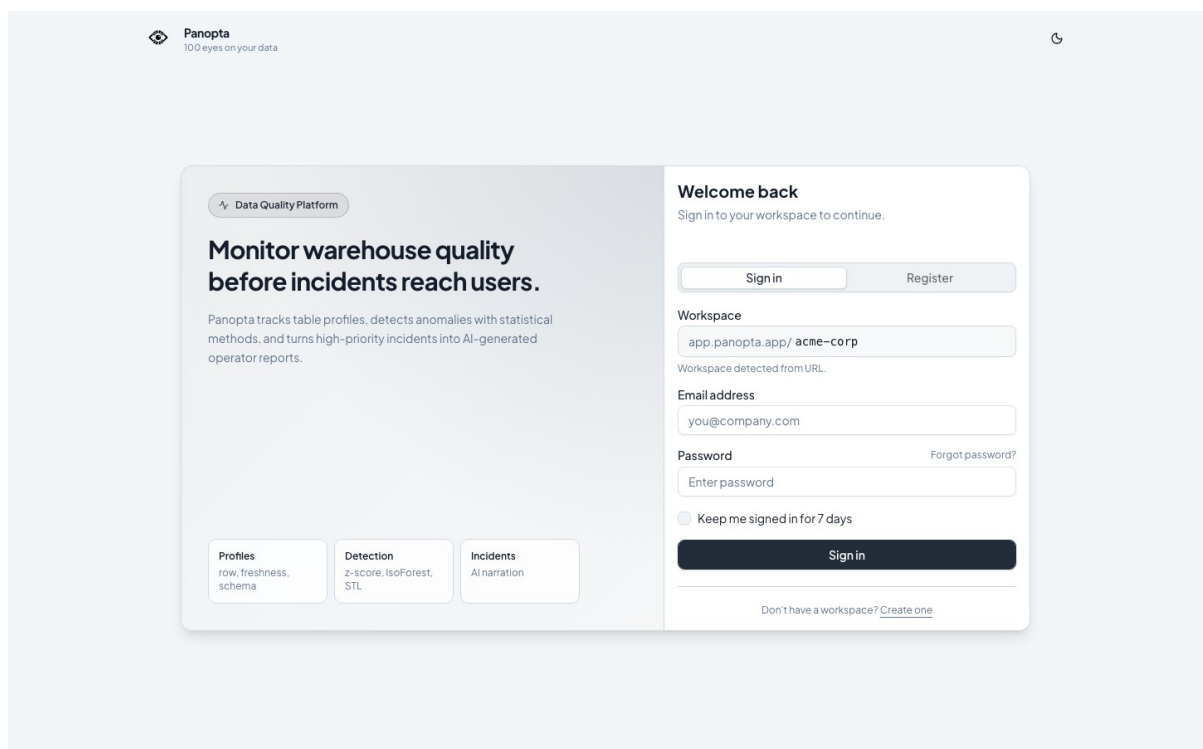


Figure 4.1 — Connexion au workspace Acme Corp

Le champ Workspace matérialise l’isolation multi-tenant avant même l’authentification. L’utilisateur choisit ensuite connexion ou inscription ; aucune vue opérationnelle n’est accessible sans contexte d’organisation explicite.

La figure 4.2 présente la première vue obtenue après authentification sur le jeu de démonstration Acme.

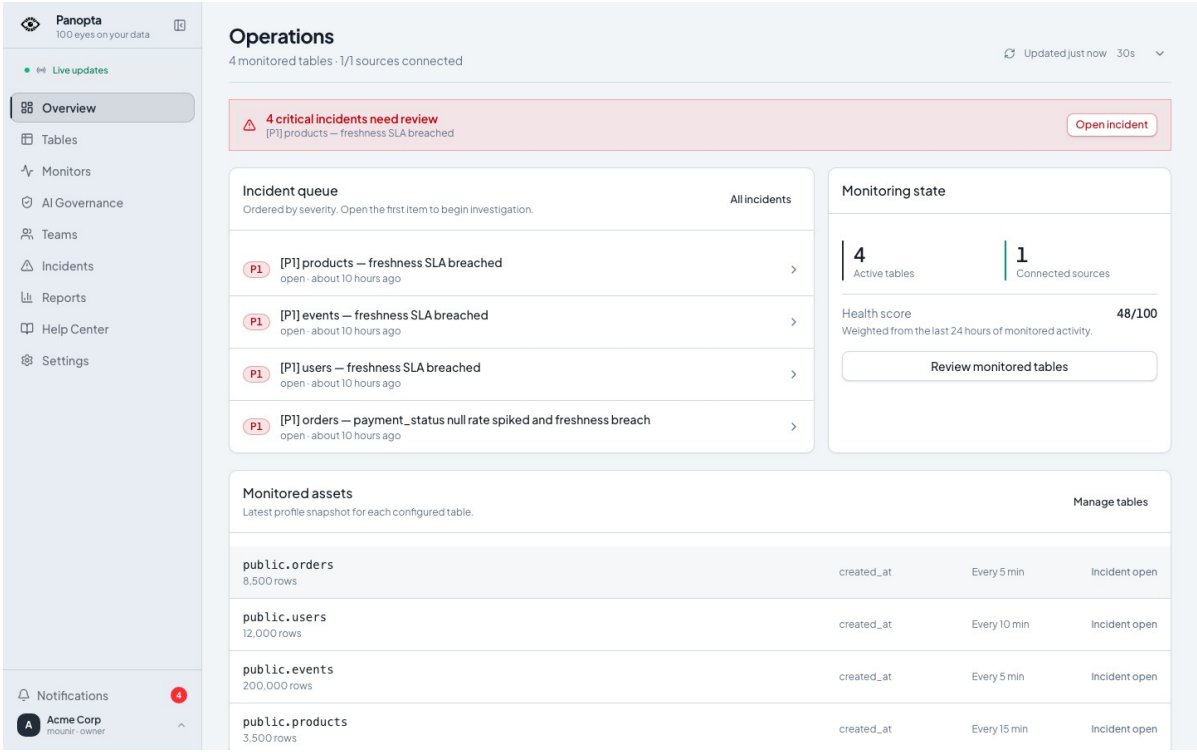


Figure 4.2 — Vue Opérations et incidents prioritaires

La bannière signale quatre incidents critiques. La file les classe par priorité, tandis que le panneau latéral résume quatre tables actives, une source connectée et un score de santé de 48 sur 100 : l’écran conduit immédiatement vers l’actif qui exige une action.

La figure 4.3 inventorie les quatre tables surveillées de la source Acme Shop DB.

Panopta
100 eyes on your data

Live updates

Overview

Tables

Monitors

AI Governance

Teams

Incidents

Reports

Help Center

Settings

Notifications

Acme Corp
monitor - owner

Tables
4 of 4 monitored tables

Updated just now 30s

+ Add table

TOTAL TABLES
4
Monitored objects

HEALTHY
0
No active signal

INCIDENTS
4
Unresolved incidents

LAST PROFILED
9h ago
9/6/2026, 10:41:26 PM

Search table name

All sources

All statuses

Name

Table	Source	Count	Last profile	Status	Actions
public.events	Acme Shop DB (live)	200k	9h ago	Incident	Run now
public.orders	Acme Shop DB (live)	8.5k	9h ago	Incident	Run now
public.products	Acme Shop DB (live)	3.5k	9h ago	Incident	Run now
public.users	Acme Shop DB (live)	12k	9h ago	Incident	Run now

Figure 4.3 — Catalogue des tables surveillées

Chaque ligne expose volumétrie, dernière exécution, état et action manuelle. Les quatre badges Incident correspondent à l'état volontairement dégradé du seed ; ils donnent une matière stable à la démonstration et ne décrivent pas une exploitation réelle.

La page Opérations constitue le point d'entrée de la démonstration. Elle affiche la file d'incidents triée par sévérité, le nombre de tables actives, les sources connectées, le score de santé et le dernier profil de chaque actif.

4.4.2 Détail de l'incident P1

L'enquête tient sur trois plans : la liste situe l'urgence, le détail rassemble les faits horodatés, puis la narration IA propose des pistes clairement séparées des mesures. L'opérateur garde la décision — affecter, acquitter, résoudre ou documenter un faux positif.

La figure 4.4 recentre l'investigation sur la liste filtrable des incidents.

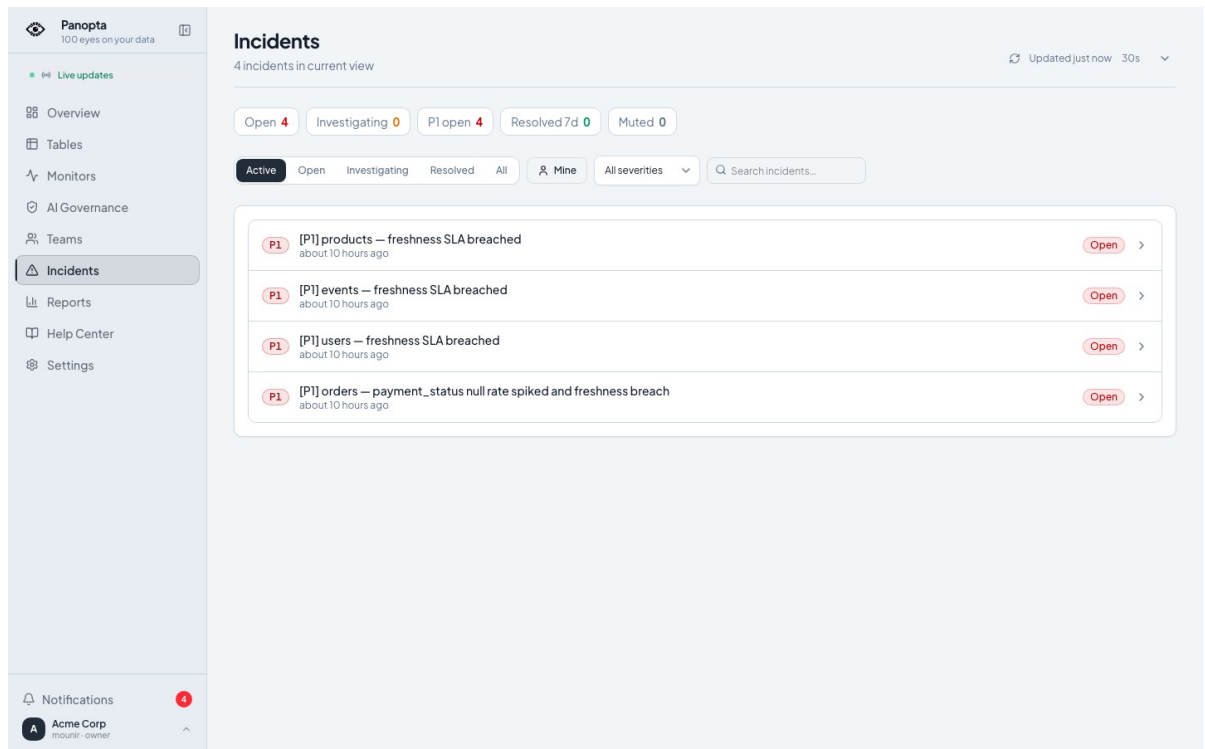


Figure 4.4 — Liste filtrable des incidents

Les onglets séparent ouverts, en cours d'analyse, résolus et ignorés ; la recherche et les filtres de sévérité réduisent la file. Les quatre lignes P1 visibles proviennent du même instantané de démonstration que la vue Opérations.

La figure 4.5 ouvre l'incident P1 de public.orders, déclenché par une hausse du taux de valeurs nulles de payment_status et une rupture de fraîcheur.

The screenshot displays the Panopta incident management interface. On the left is a sidebar with navigation options: Overview, Tables, Monitors, AI Governance, Teams, Incidents (selected), Reports, Help Center, and Settings. The main content area shows the details of incident P1, titled "[P1] orders — payment_status null rate spiked and freshness breach". The incident is marked as "Open" and "P1". Action buttons include "Acknowledge", "Resolve" (highlighted), "Mute 24h", and "False positive". It was detected on 9/6/2026 at 10:41:28 PM, about 10 hours ago.

Status timeline:

- Detected:** 9/6/2026, 10:41:28 PM
- Acknowledged:** Pending
- Investigating:** Pending
- Resolved:** Pending

AI incident analysis: High Confidence

In the public.orders table, the null rate for payment_status jumped from ~0.02% to 100% (observed=1.0) and the freshness metric showed a breach of 7,526,251 seconds (~87 days) on 2026-09-06.

Likely causes:

- High:** A faulty batch load job incorrectly set payment_status to NULL and reset other columns to default values, causing low cardinality and stale timestamps in public.orders.
- Medium:** A change in the ingestion pipeline stopped updating the created_at column and began inserting rows with NULL payment_status, leading to the observed null spike and freshness lag.
- Low:** A database replication or failover event caused the orders table to revert to an outdated snapshot lacking payment_status values.

Affected table:

- Name: public.orders
- Monitoring: Monitoring active
- Source: Source name unavailable
- Technical identifiers: View table detail

Assignment:

- Assignee: Assign to user...
- Team: No team

Impact assessment:

Payment-related dashboards, revenue reporting, fraud detection, and order fulfillment processes are

Figure 4.5 — Détail de l'incident critique orders

La chronologie, les faits, la table affectée et l'affectation sont réunis sans masquer l'origine du signal. Les boutons Acquitter et Résoudre modifient l'état de travail ; ils ne corrigent pas automatiquement la donnée source.

La figure 4.6 agrandit la narration IA associée au même incident.

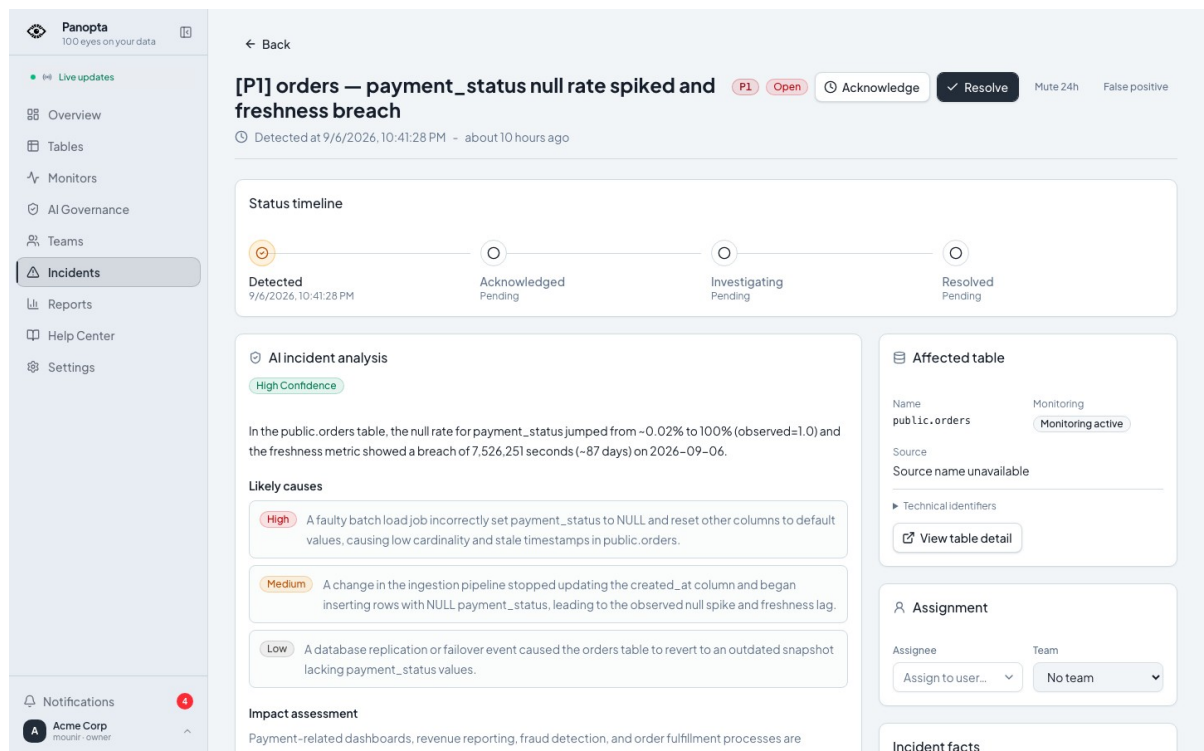


Figure 4.6 — Analyse IA de l'incident et actions proposées

Le texte distingue résumé, causes probables et impact, avec des niveaux de confiance visibles. Ces propositions accélèrent l'enquête, mais restent des hypothèses à confronter aux métriques et au système source avant toute décision.

La fiche incident réunit la chronologie, les contrôles échoués, les valeurs observées, l'analyse IA, les causes probables, les actions recommandées et une requête de diagnostic copiable. Les hypothèses générées restent distinctes des faits mesurés.

4.4.3 Détail de la table surveillée

La fiche public.orders relie série temporelle, contrôles et configuration. Les recommandations ne s'activent pas en silence : elles conduisent vers le catalogue des moniteurs et un constructeur DSL qui expose définition, mode et validation.

La figure 4.7 montre le profil courant de public.orders et son historique récent.

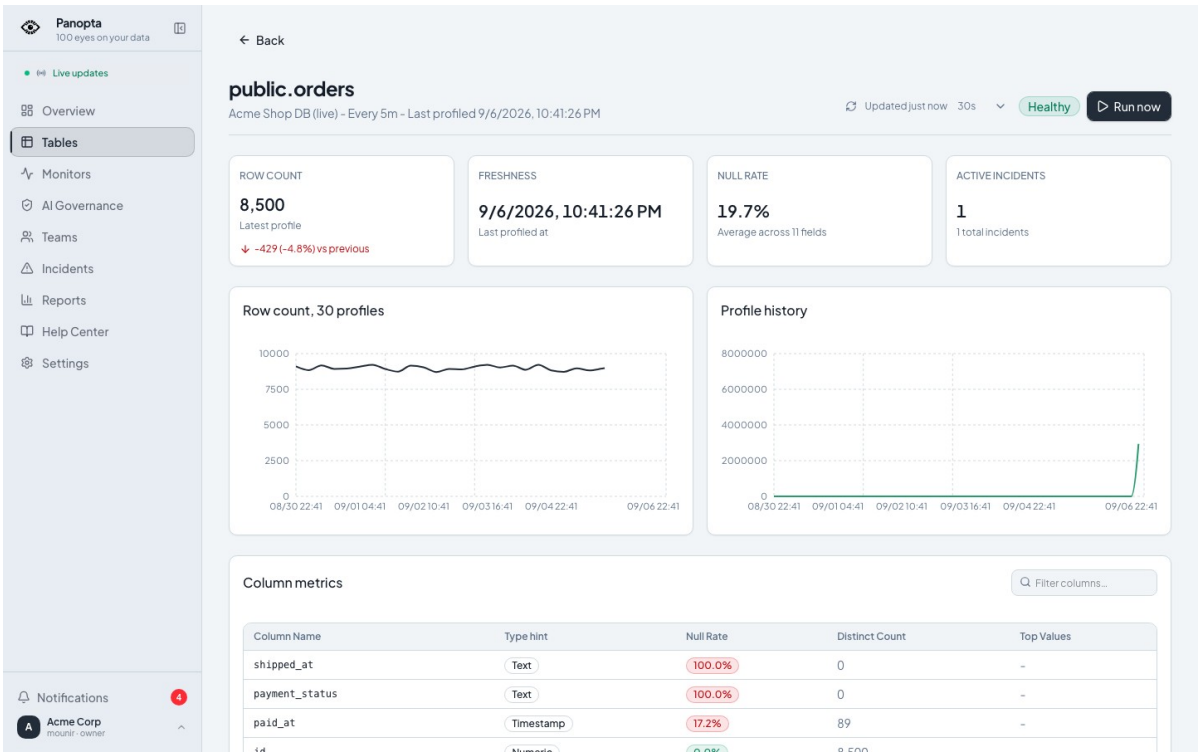


Figure 4.7 — Profil de la table public.orders

Le seed affiche 8 500 lignes, une fraîcheur datée, un taux de nullité agrégé et un incident actif. Les graphiques replacent ces valeurs dans le temps ; le tableau de colonnes permet ensuite de localiser la dérive plutôt que de s’arrêter au score global.

La figure 4.8 poursuit la même fiche avec les dérives de percentiles, exclusions de colonnes, routes d'alerte et recommandations de moniteurs.

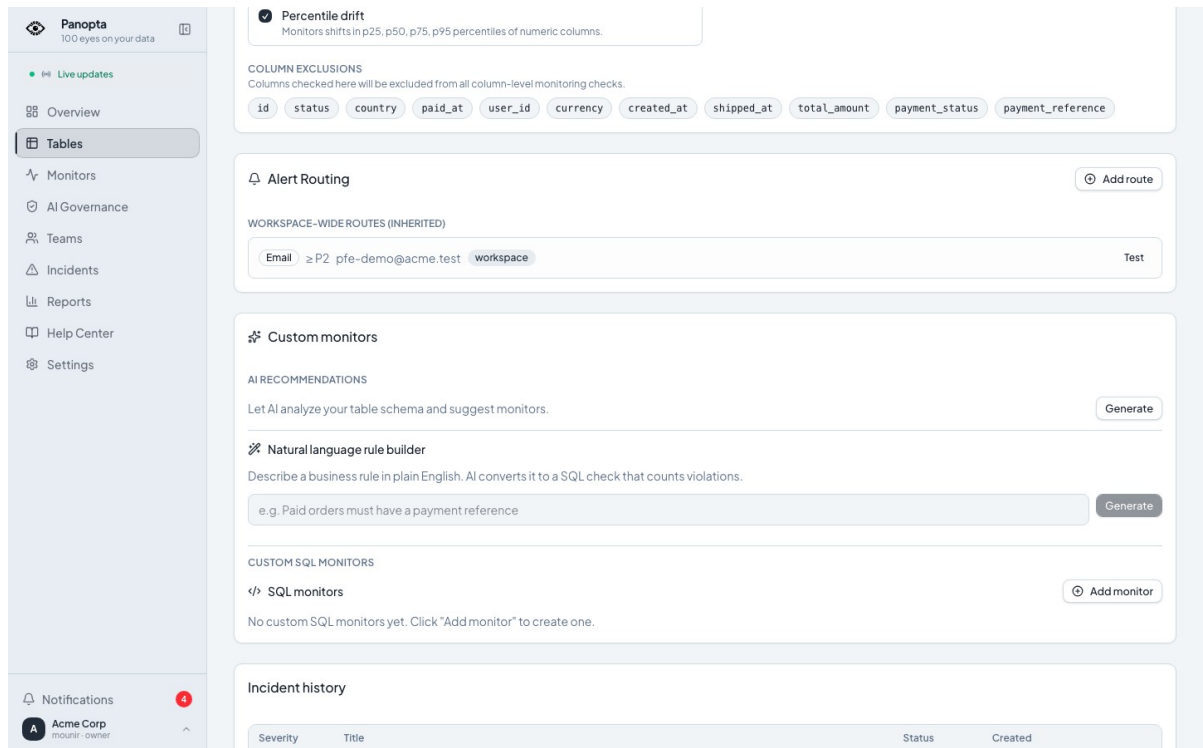


Figure 4.8 — Recommandations de moniteurs pour public.orders

Une recommandation ne devient pas silencieusement un contrôle actif. L'utilisateur la relit, choisit sa portée, puis ouvre le constructeur ; ce passage protège le catalogue contre des règles opaques générées sans validation humaine.

La figure 4.9 présente le catalogue des moniteurs typés avant la création d'une première règle dans ce seed.

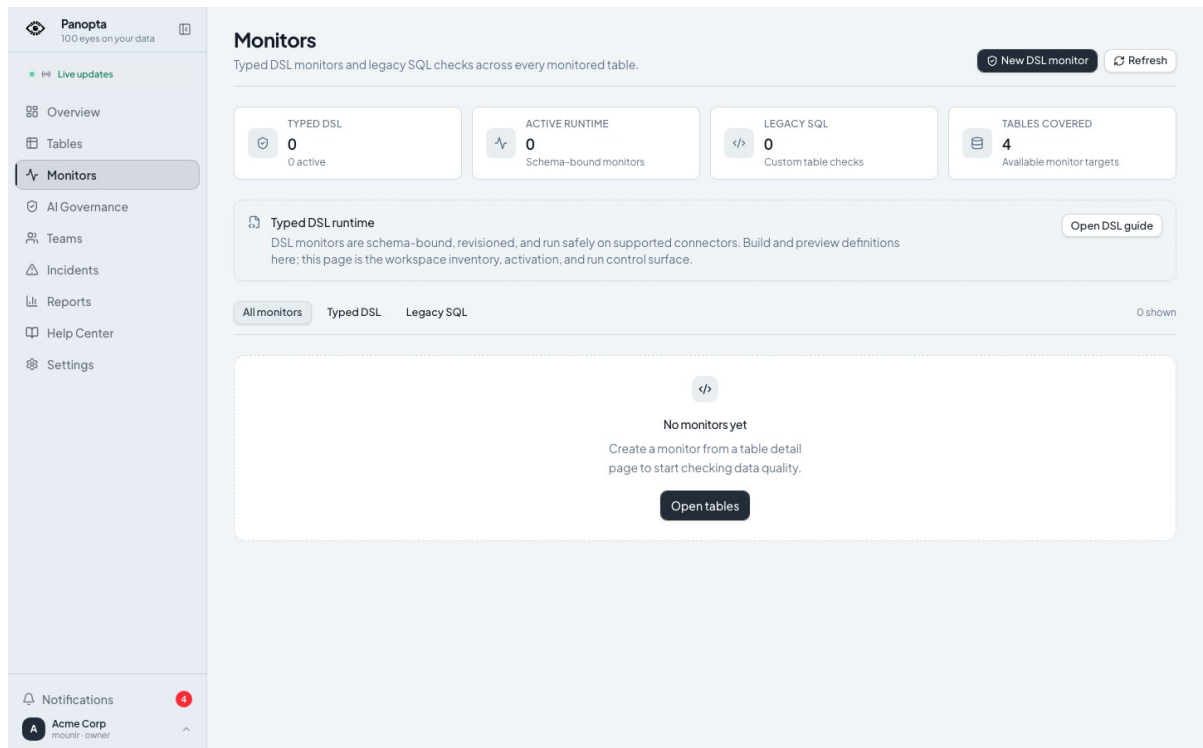


Figure 4.9 — Catalogue des moniteurs typés

Les compteurs à zéro sont intentionnels et rendent la limite visible. L'écran sépare moniteurs DSL et contrôles SQL historiques, rappelle le principe de révision liée au schéma et propose un point d'entrée unique pour la création.

La figure 4.10 ouvre le constructeur d'un moniteur de volume sur public.orders.

New typed DSL monitor
Build a schema-bound monitor, preview its connector plan, then activate the immutable revision.

Monitored table
public.orders

Monitor name
orders-row-count
Stored as: lowercase-kebab-name

Rule type
Volume - row count

Breach when row count is
greater than

Row count threshold
0

Monitor description
What should this monitor protect?

Owner
data-platform@example.com

Quality dimension
Not specified

Severity
P2 - High

Run mode
Alert on breach

Trigger
After each profile

Consecutive breaches
1

Recovery passes
1

Cancel Validate & preview

Figure 4.10 — Constructeur de moniteur DSL

Le formulaire rend explicites le type de règle, le seuil, la sévérité, le mode d'exécution, le déclencheur et les conditions de récupération. Valider et prévisualiser précède l'activation afin que la définition soit contrôlée avant de devenir une révision immuable.

Cette vue relie l'incident à l'actif concerné. Elle permet de présenter l'historique des profils et l'évolution des métriques de qualité avant et après l'anomalie.

4.4.4 Alertes et gouvernance IA

4.4.4.1 Rapports, équipes et astreintes

La réponse à l'incident dépasse l'écran technique. Les rapports condensent une semaine ; les équipes rendent l'affectation concrète ; les créneaux d'astreinte indiquent qui peut réellement recevoir une escalade.

La figure 4.11 synthétise la semaine seedée sous forme d'indicateurs de santé et de contrôles en échec.

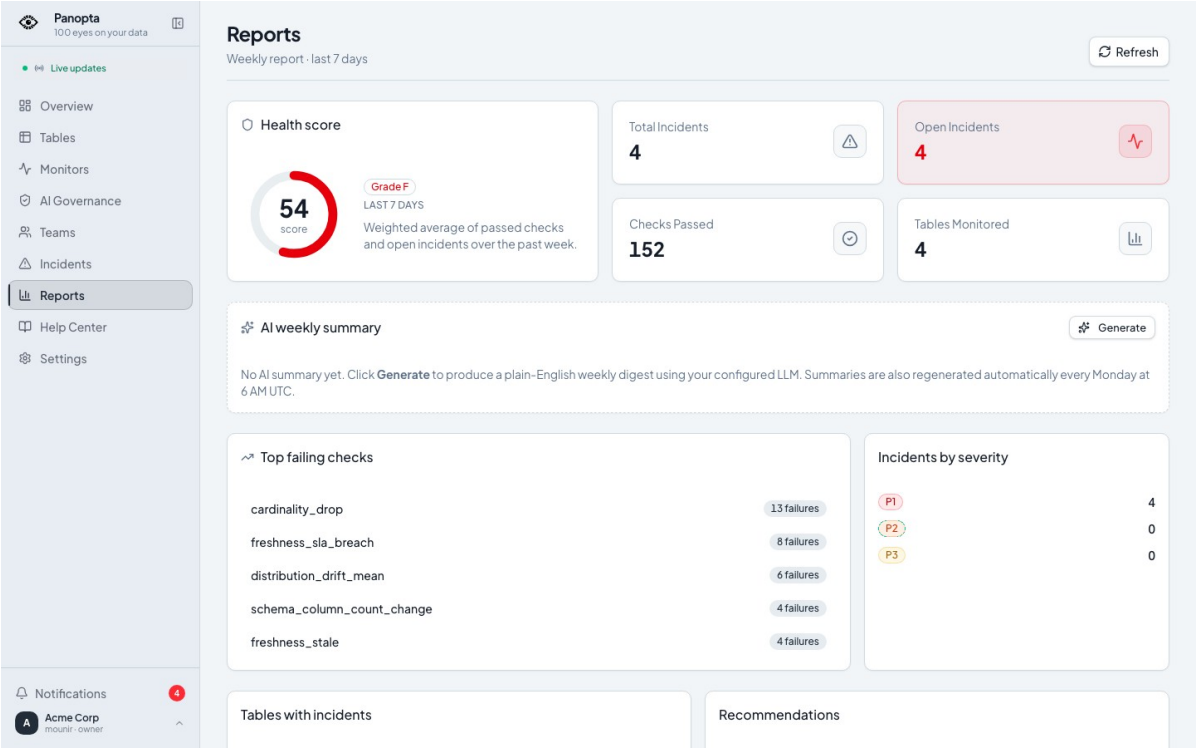


Figure 4.11 — Rapports hebdomadaires de santé

Le score de 54, les quatre incidents ouverts, les 152 contrôles réussis et les quatre tables surveillées partagent le même intervalle. La zone de résumé IA reste vide tant que l'utilisateur ne demande pas sa génération, ce qui évite de présenter un texte absent comme une preuve.

La figure 4.12 recense les équipes créées dans le workspace Acme.

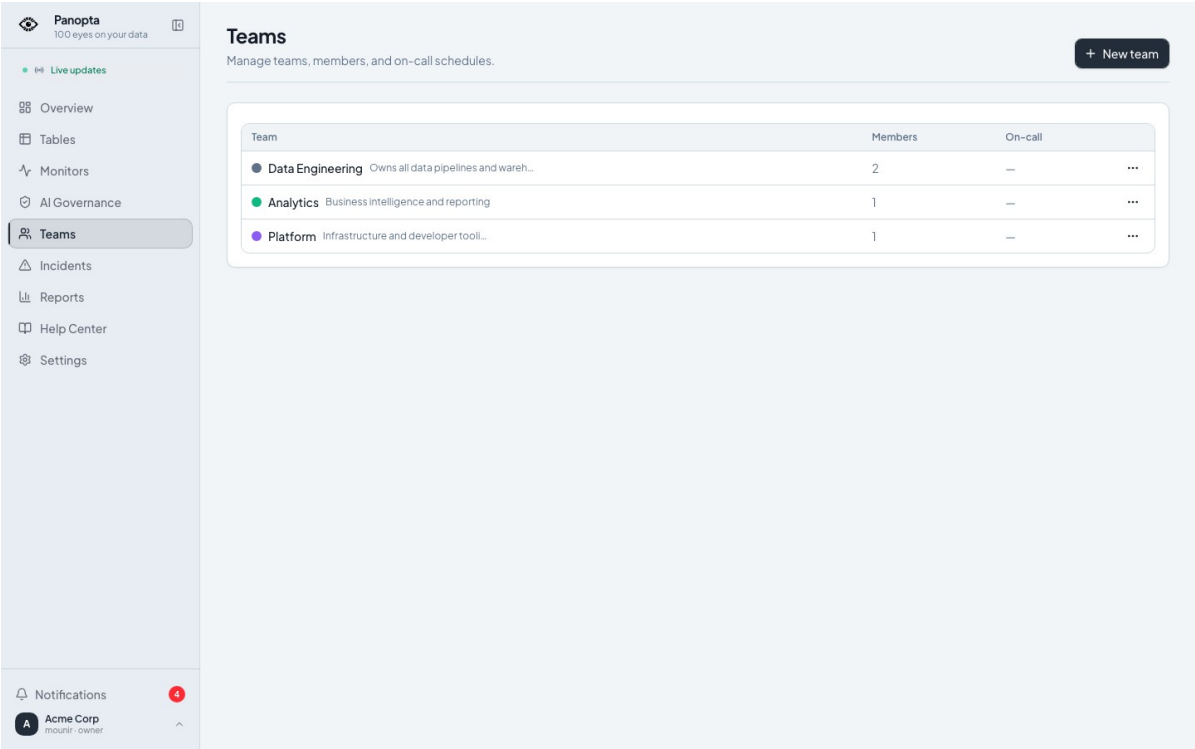


Figure 4.12 — Répertoire des équipes

Data Engineering, Analytics et Platform possèdent des membres et responsabilités distinctes. Cette vue transforme une simple liste d'utilisateurs en structure d'affectation exploitable pour les incidents et les astreintes.

La figure 4.13 ouvre le détail de l'équipe Data Engineering.

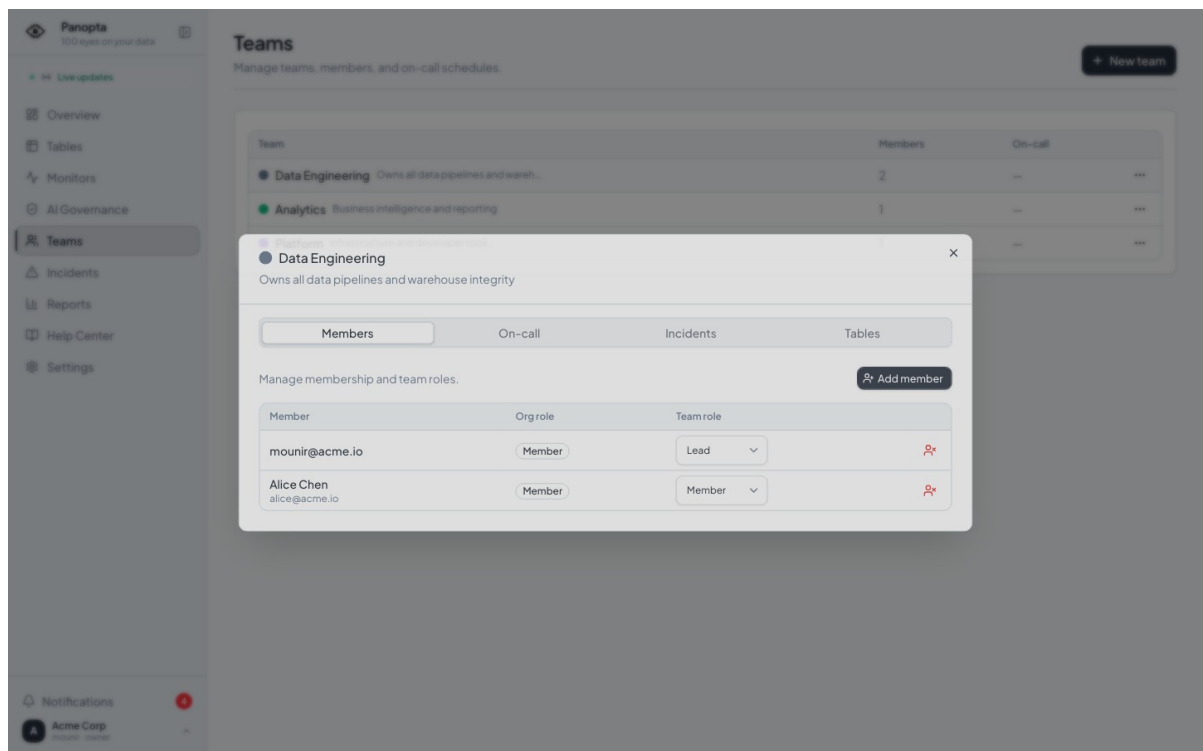


Figure 4.13 — Membres de l'équipe Data Engineering

L'onglet Membres expose les rôles au sein de l'équipe et permet leur modification sans changer le rôle global dans l'organisation. Les autres onglets relient la même équipe aux astreintes, incidents et tables dont elle a la charge.

4.4.4.2 Sources, connecteurs et notifications

Le catalogue distingue les capacités annoncées de chaque moteur. La source seedée sert de verticale vérifiée ; le routage et les préférences montrent ensuite comment la même alerte se distribue sans imposer un canal unique.

La figure 4.14 affiche les sources enregistrées dans les paramètres du workspace.

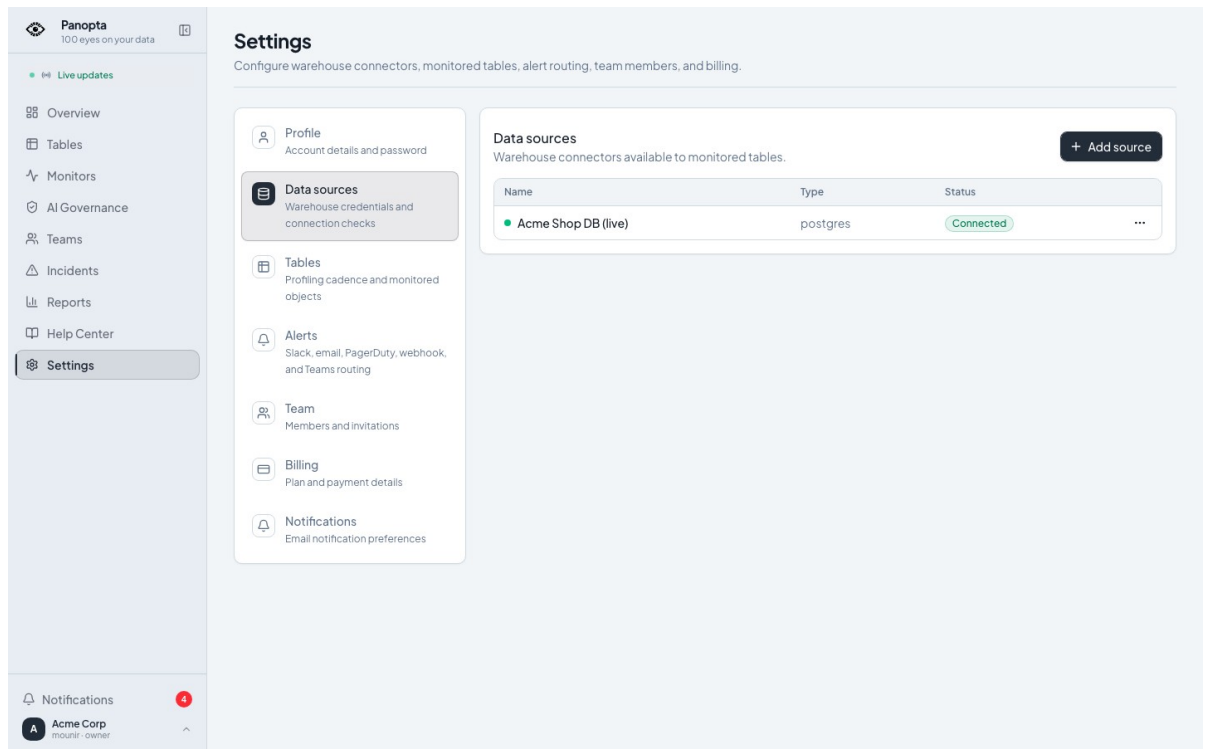


Figure 4.14 — Sources de données enregistrées

Acme Shop DB est la seule source connectée du scénario et utilise PostgreSQL. Ce choix borne clairement la preuve d'intégration : le catalogue propose davantage de moteurs, mais cette capture n'atteste que la verticale réellement démarrée.

La figure 4.15 montre le formulaire d'ajout d'une source et la sélection du connecteur PostgreSQL.

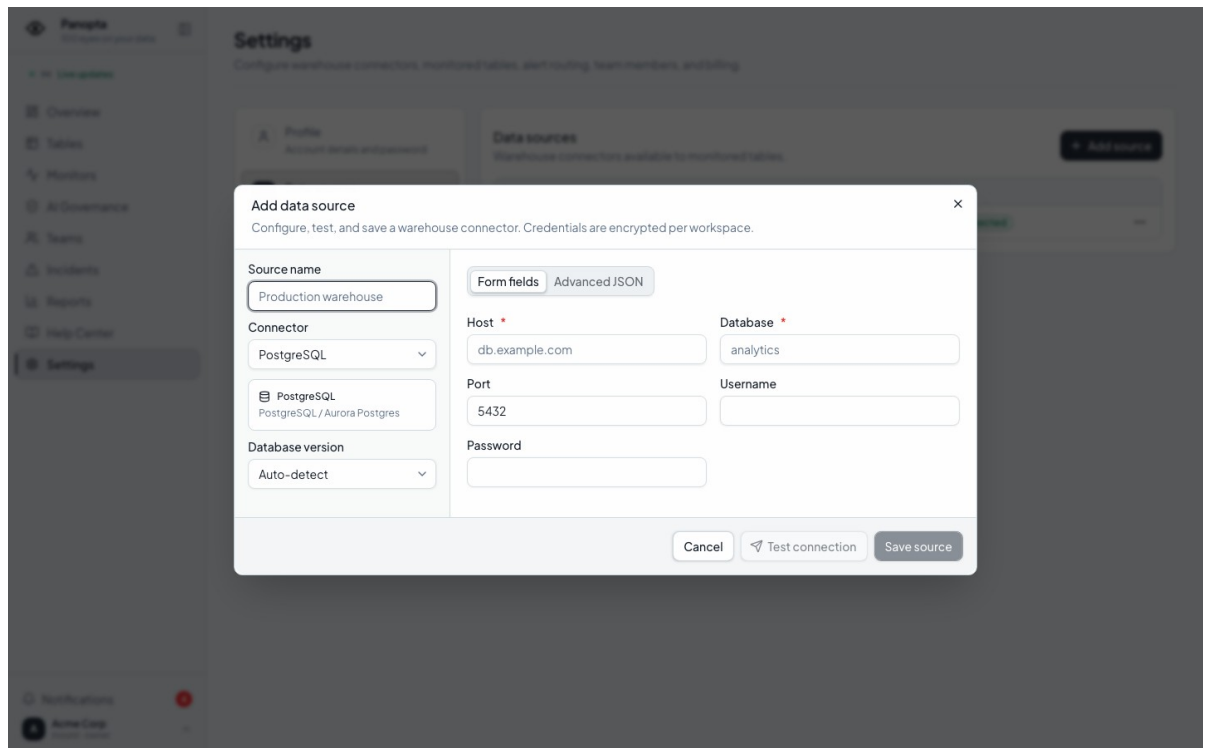


Figure 4.15 — Catalogue des connecteurs disponibles

Hôte, port, base, utilisateur et mot de passe sont saisis avant le test de connexion. Le mot de passe n'est jamais réaffiché après enregistrement ; le backend chiffre la configuration dans le contexte du tenant.

La figure 4.16 présente les canaux disponibles et la route e-mail utilisée pendant la démonstration.

The screenshot shows the Panopta Settings interface. On the left is a sidebar with navigation links: Overview, Tables, Monitors, AI Governance, Teams, Incidents, Reports, Help Center, and Settings (highlighted). The main content area is titled 'Settings' with a subtitle 'Configure warehouse connectors, monitored tables, alert routing, team members, and billing.' Below this is a list of settings categories: Profile, Data sources, Tables, Alerts (selected), Team, Billing, and Notifications. The 'Alerts' section is expanded, showing 'Alert routes' configuration. It includes a '+ Add alert' button and a list of available channels: Email, Slack, Generic webhook, PagerDuty, Microsoft Teams, and OpsGenie. Each channel has a brief description and an 'Available' status. Below the list is a table showing the configured alert route.

Channel	Destination	Scope	Minimum severity
Email	pfe-demo@acme.test	All workspace incidents	P2

Figure 4.16 — Routes d'alerte par canal et sévérité

La route `pfe-demo@acme.test` couvre tous les incidents du workspace à partir de P2. Slack, PagerDuty et les webhooks restent visibles comme capacités de routage, sans laisser croire que chacun a été validé dans cette session locale.

La figure 4.17 descend au niveau des préférences de notification de l'utilisateur connecté.

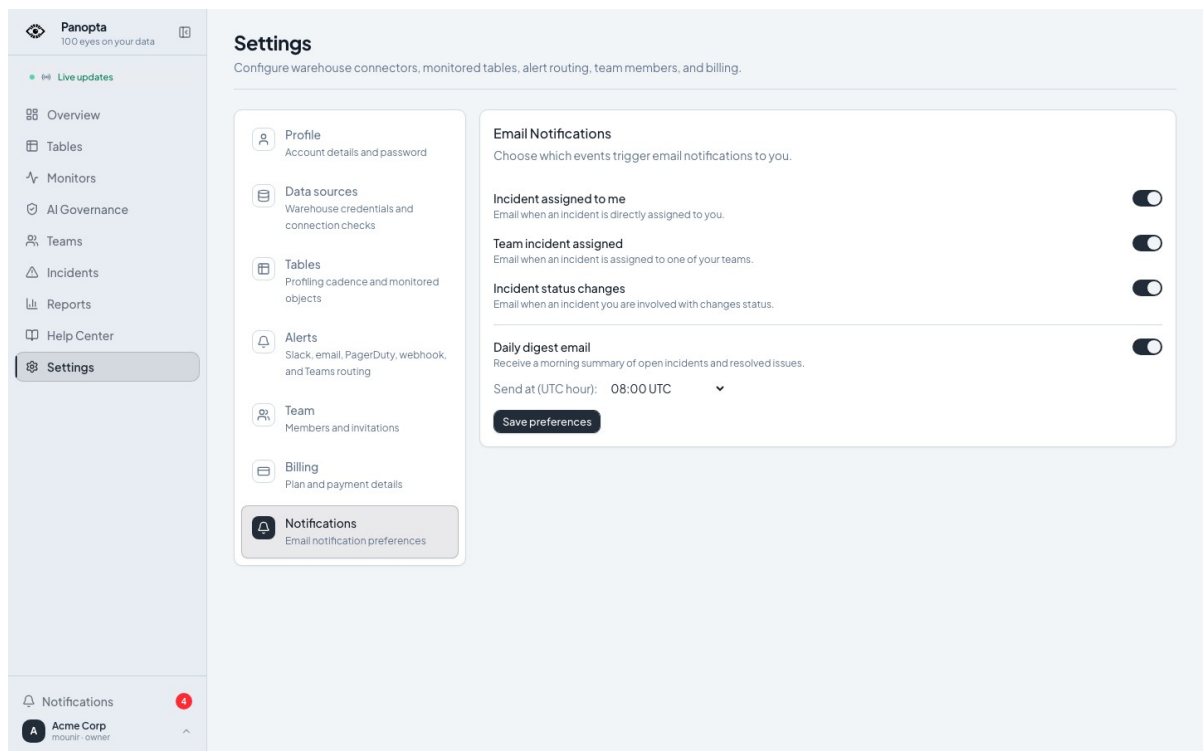


Figure 4.17 — Préférences individuelles de notification

Les bascules distinguent attribution, activité d'équipe et changement d'état ; le digest quotidien possède sa propre heure et son fuseau. Une route d'organisation définit donc la livraison possible, tandis que ces préférences règlent ce que chaque personne souhaite recevoir.

4.4.4.3 Registre de gouvernance IA

La gouvernance IA désigne ici l'ensemble des responsabilités, versions, déclarations de données et preuves nécessaires pour répondre à cinq questions simples : quel système fonctionne, dans quel but déclaré, avec quelles données, sous la responsabilité de qui, et avec quels contrôles disponibles ? DataWatch matérialise ces réponses dans un registre isolé par organisation. Les versions, manifestes et évaluations terminales sont immuables ; les états inconnus ou périmés restent visibles au lieu d'être maquillés en succès.

Dans le scénario Acme, un assistant RAG de support est lié à une table surveillée et à une empreinte de schéma. À chaque profil réussi, DataWatch peut réévaluer l'âge des preuves, la fraîcheur du schéma, la présence de responsables, les rôles d'accès déclarés et quelques incohérences vectorielles. L'écran rassemble ensuite état global, confiance des preuves, risque résiduel, raisons de contrôle et chronologie. Il aide un opérateur à voir ce qui manque. Rien de plus.

La figure 4.18 ouvre la file de travail du prototype de gouvernance IA.

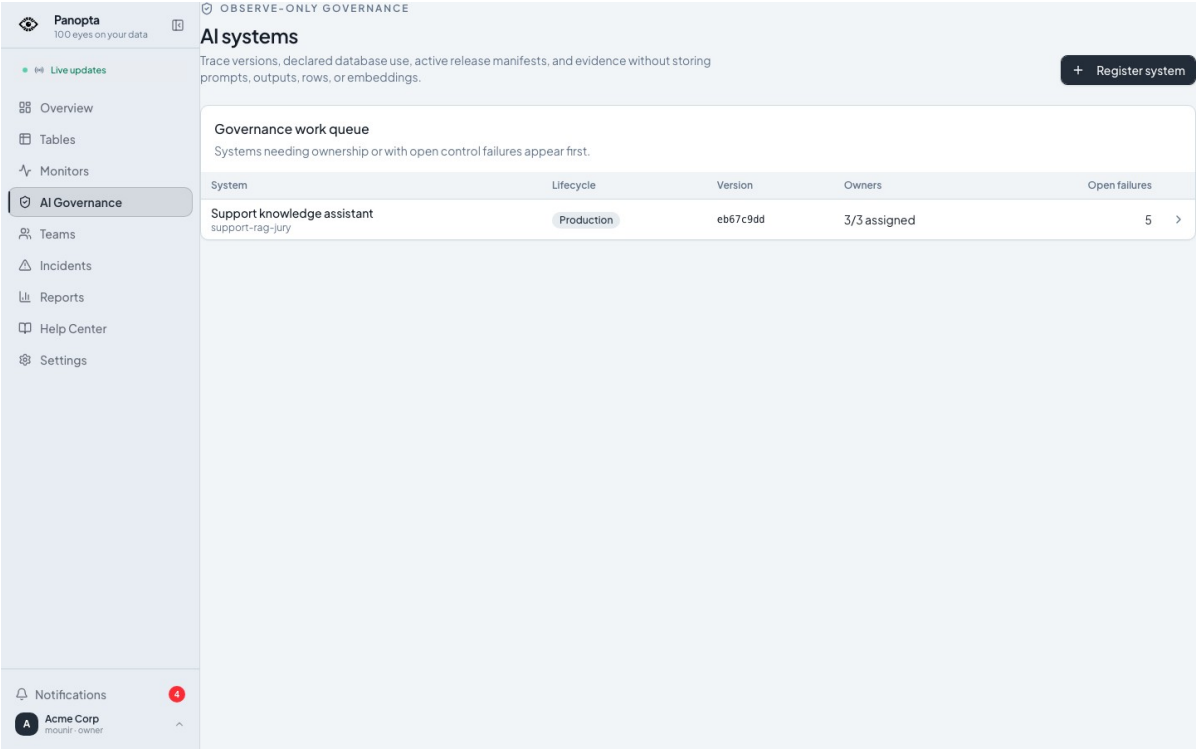


Figure 4.18 — File de travail de gouvernance IA

Un seul système seedé apparaît : Support knowledge assistant, avec trois responsables sur trois et cinq contrôles ouverts. Cette faible volumétrie est volontaire ; elle démontre le registre et ses états, pas sa capacité à gouverner un portefeuille industriel.

La figure 4.19 détaille l'état calculé pour le système IA de support.

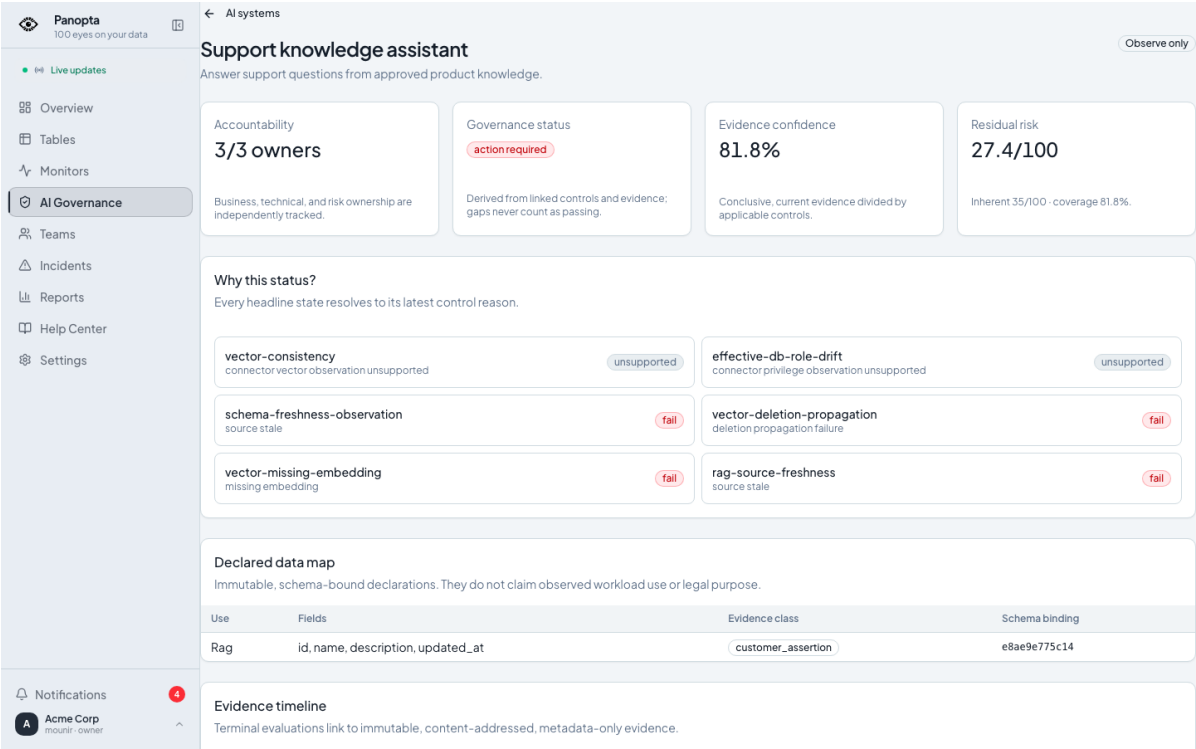


Figure 4.19 — Détail d'un système IA déclaré

L'écran affiche une action requise, une confiance des preuves de 81,8 % et un risque résiduel de 27,4 sur 100. Les cartes de contrôle exposent leurs raisons et leurs échecs ; ces nombres appartiennent au scénario seedé et ne sont ni un score réglementaire ni une certification.

La figure 4.20 prolonge cette fiche par la chronologie des preuves attachées au système.

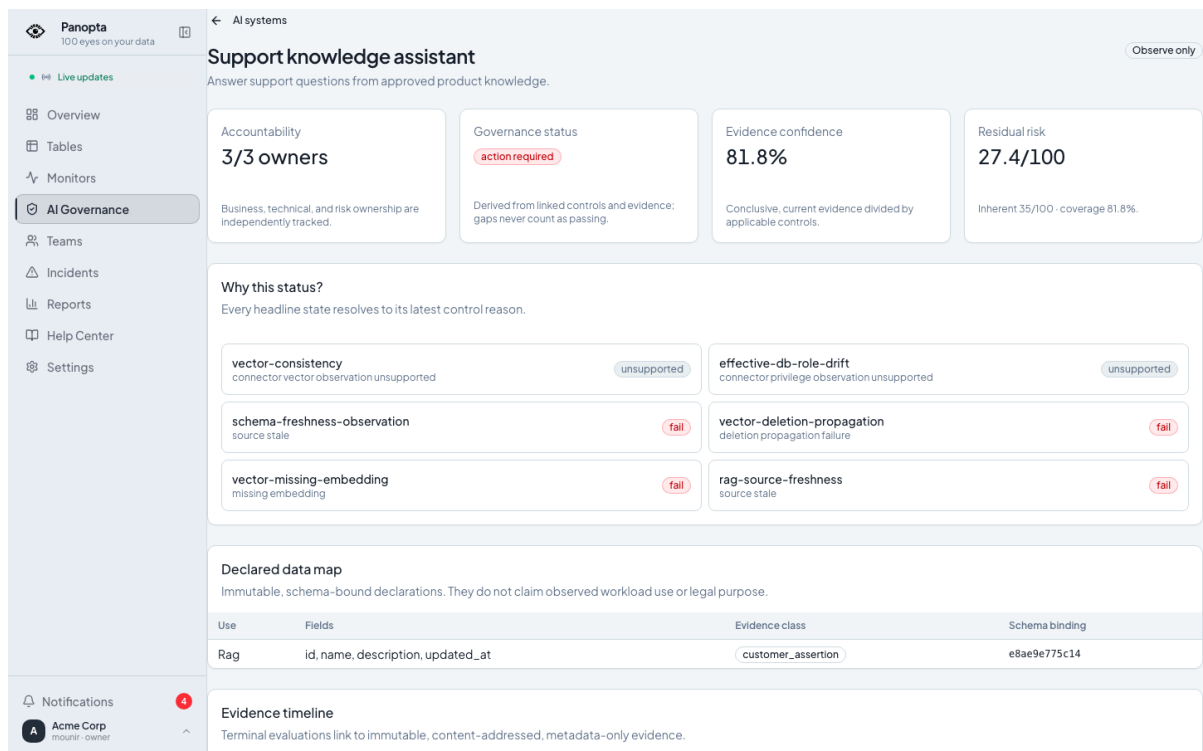


Figure 4.20 — Chronologie des preuves de gouvernance

Chaque entrée conserve type, provenance, validité et empreinte afin de rendre une évaluation rejouable. Le registre ne stocke ni prompts ni réponses métier : la chronologie porte uniquement sur des métadonnées techniques bornées.

4.4.4.4 Limites du prototype de gouvernance IA

Cette brique est une preuve de concept, volontairement petite. La démonstration repose sur un seul système IA seedé, une verticale PostgreSQL/pgvector et des anomalies préparées pour être reproductibles. Plusieurs informations demeurent des déclarations client ; elles ne prouvent ni l'usage réel des données, ni la finalité juridique, ni le comportement effectif du modèle. Le prototype ne mesure pas la robustesse, les biais, l'équité, la sécurité adversariale ou la dérive des sorties. Il ne lit d'ailleurs ni prompts, ni réponses, ni lignes métier : le registre conserve uniquement des métadonnées bornées.

Le mode observe-only est la frontière la plus importante. Un contrôle en échec ouvre un signal et peut déclencher une alerte, mais il n'empêche pas une release. L'interface actuelle permet surtout d'enregistrer un système puis de consulter le scénario préparé ; la création complète des versions, usages, manifestes, approbations et politiques n'est pas encore un parcours SaaS abouti. Pour devenir une fonction produit crédible, il faudra un éditeur de bout en bout, des gates d'approbation configurables, un véritable circuit de revue humaine, des exports d'audit, une politique de rétention, des contrôles multi-fournisseurs et des validations répétées sur des cas non seedés.

4.4.4.5 Portail staff

L'administration de la plateforme utilise une authentification séparée. Le staff voit les organisations, les usages et l'état d'un tenant ; il n'emprunte pas l'identité d'un membre du workspace.

La figure 4.21 montre la porte d'entrée distincte du portail réservé au personnel DataWatch.

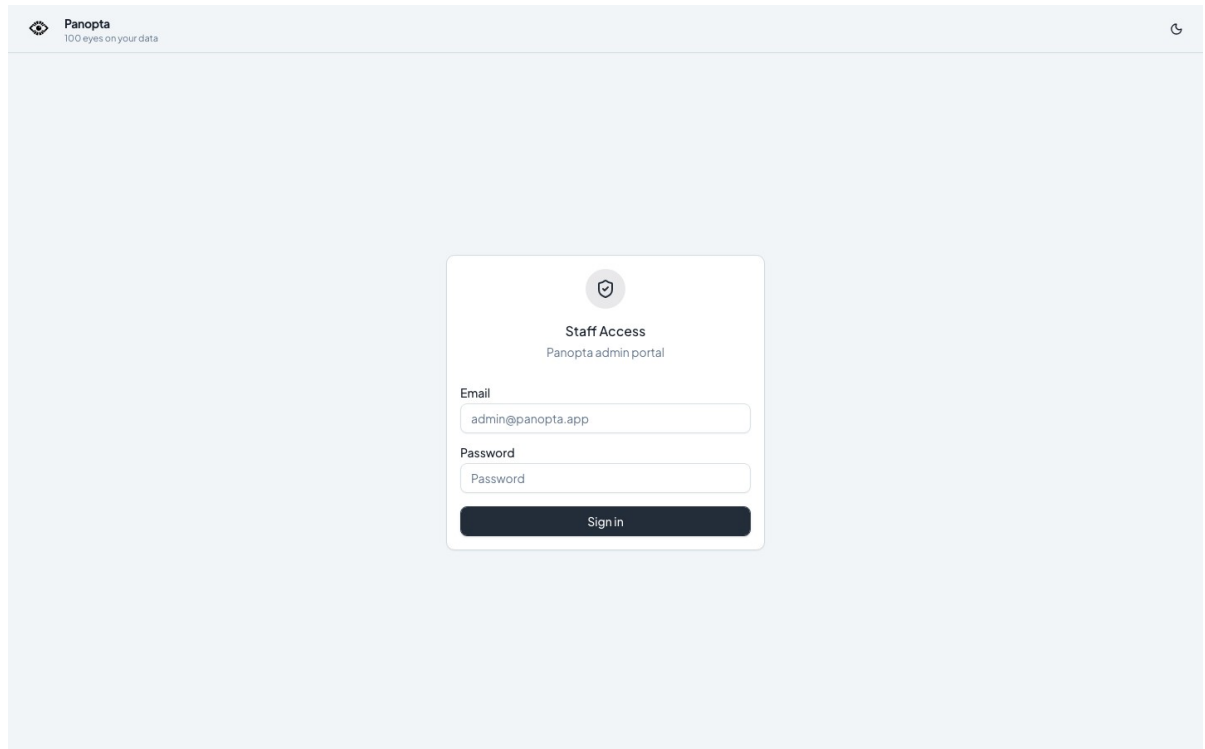


Figure 4.21 — Connexion isolée au portail staff

L'absence de champ Workspace n'est pas un oubli : les comptes staff appartiennent à un domaine d'identité séparé. Une session client ne peut donc pas être recyclée pour ouvrir les fonctions d'administration globale.

La figure 4.22 présente l’inventaire des organisations depuis le portail staff.

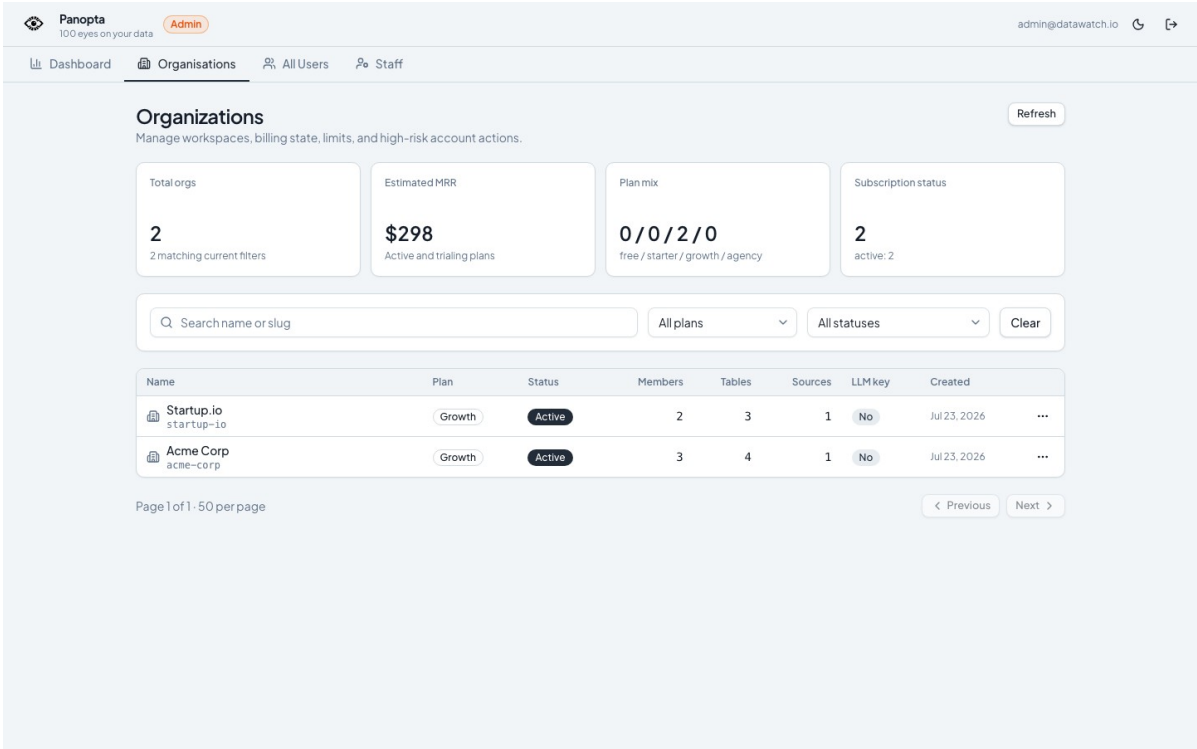


Figure 4.22 — Administration des organisations clientes

Le seed contient deux tenants actifs, cinq membres au total et un revenu mensuel estimé à 298 dollars. Recherche, filtres de plan et statut précèdent l’ouverture d’une organisation ; les valeurs restent des données de démonstration.

La figure 4.23 rassemble les indicateurs commerciaux et opérationnels visibles par le staff.

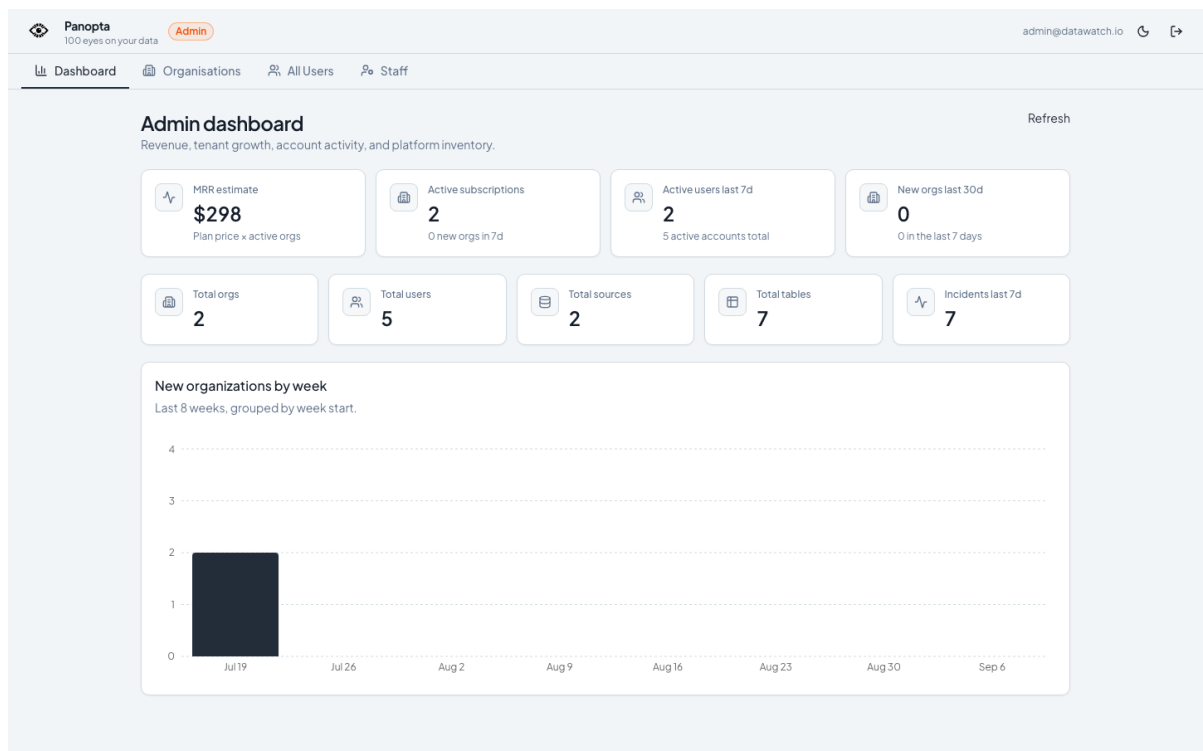


Figure 4.23 — Tableau de bord global du portail staff

Le tableau de bord sépare revenu récurrent, abonnements, activité récente, organisations, utilisateurs, sources, tables et incidents. Le graphique hebdomadaire contextualise le stock par un flux de créations, sans prétendre constituer une comptabilité de production.

La figure 4.24 ouvre la fiche administrative d'Acme Corp.

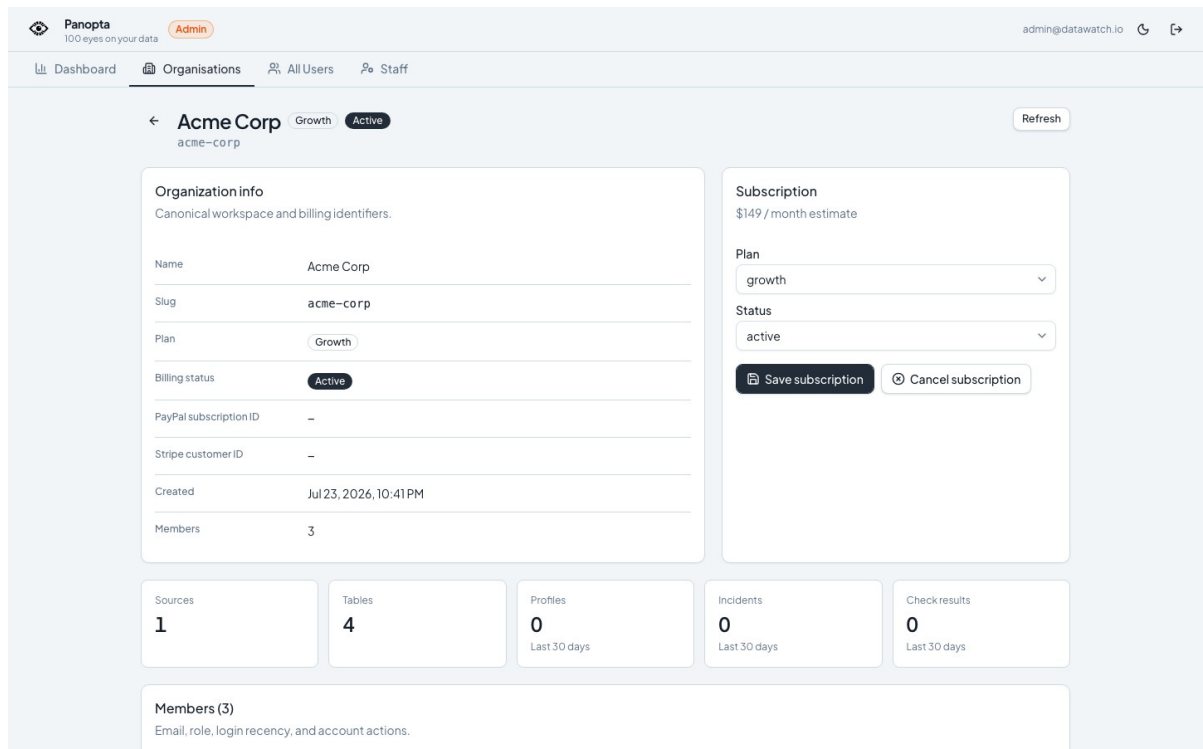


Figure 4.24 — Détail opérationnel de l'organisation Acme Corp

Le plan Growth actif est affiché à 149 dollars par mois, avec une source, quatre tables et trois membres. Les compteurs sur trente jours permettent de vérifier l'activité déclarée avant une modification d'abonnement ; ils restent isolés des écrans opérationnels du client.

La démonstration se poursuit dans Paramètres → Alertes avec la route email P1/P2, puis dans MailHog pour la preuve de livraison. La page Gouvernance IA présente un prototype observe-only : inventaire, usages déclarés, provenance des preuves et raisons des contrôles. Le scénario est restreint à une verticale PostgreSQL/pgvector seedée ; il sert à éprouver le modèle de données et l'interface, pas à prouver une gouvernance industrielle.

4.4.5 Règles de présentation des captures

Chaque capture doit être introduite dans le texte, numérotée par chapitre et suivie d'une légende descriptive. Les mots de passe, clés, identifiants sensibles et données non publiques doivent être masqués. Utiliser une largeur homogène et éviter les captures trop longues.

4.5 Tests et validation

4.5.1 Stratégie de test

La stratégie combine plusieurs niveaux. Les tests unitaires couvrent les règles, validateurs et transformations pures. Les tests de service vérifient les transactions, autorisations et états. Les tests d'intégration nécessitent PostgreSQL et Redis afin d'exposer les erreurs que les

doubles de test masquent. Enfin, Playwright rejoue les parcours visibles sur une pile seedée. La construction frontend et la validation de la configuration Docker complètent ces preuves.

4.5.2 Scénarios fonctionnels

Tableau 4.2 — Matrice de validation fonctionnelle

Scénario	Résultat attendu	Preuve
Connexion Acme	Accès limité au workspace	Parcours navigateur
Ouverture de l'incident orders	P1, signaux et narration visibles	Capture et API
Profil de la table	Historique et métriques cohérents	Capture et base seedée
Alerte e-mail	Message acheminé vers MailHog	Interface et boîte de test
Moniteur typé	Révision validée et résultat traçable	Tests backend
Gouvernance IA	États et preuves observe-only visibles	Playwright et ledger

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

4.5.3 Résultats et interprétation

La session de validation du rapport a démarré une pile Docker, réinitialisé les données et rejoué les cinq parcours de démonstration. La suite backend a produit plusieurs centaines de succès, mais elle a également révélé des tests ignorés et des échecs dépendant du montage complet du dépôt. Ces résultats sont présentés comme une photographie reproductible de l'environnement local, et non comme une garantie universelle.

La distinction entre test syntaxique, test unitaire et preuve d'intégration est essentielle. Une configuration Docker valide ne prouve pas que les services démarrent ; une suite avec dépendances simulées ne prouve pas qu'une transaction réelle fonctionne ; une capture correcte ne prouve pas la résistance à la charge. Pour la couche IA, les tests vérifient les seuils d'activation, les scores produits, la persistance des observations et le comportement sur les anomalies seedées. Ils ne constituent pas encore une mesure de précision ou de rappel sur un corpus annoté. Le rapport conserve cette limite à proximité des résultats.

La validation combine des tests unitaires et d'intégration, une pile Docker seedée, des parcours Playwright et des contrôles de qualité de build. Le seed de démonstration crée les espaces Acme et Startup, configure une route d'alerte P1/P2 vers MailHog et attend une narration disponible pour l'incident orders avant la prise de démonstration.

VALIDATION REJOUÉE SUR LA PILE LOCALE SEEDÉE

La pile Docker est saine : l'API confirme la connexion à PostgreSQL et Redis. Le seed déterministe a créé deux espaces de travail, sept tables surveillées, 463 profils historiques et sept incidents ouverts, dont l'incident P1 « orders » avec narration disponible.

Les cinq parcours Playwright passent intégralement : création et test d'un moniteur, configuration d'une alerte, page Moniteurs avec temps réel, gouvernance IA et parcours PFE Operations → incident. Aucun de ces scénarios n'a signalé d'erreur console, d'erreur de page ou de réponse HTTP en échec.

Dans le conteneur API standard, 321 tests réussissent, 50 sont ignorés et deux contrôles échouent parce que seule la racine backend est montée. Rejouée dans la même image avec le dépôt complet disponible, la suite atteint 323 réussites et 50 tests ignorés. Le rejeu hôte strict reste distinct : Python 3.14, pytest-asyncio et plusieurs services optionnels demandent un environnement dédié. Ces limites sont consignées ; elles ne deviennent pas, par glissement de vocabulaire, une validation totale.

4.6 Discussion des limites

Le premier risque concerne la profondeur des connecteurs. Un adaptateur peut satisfaire un contrat abstrait sans avoir été validé avec des identifiants réels, des volumes représentatifs et les particularités de son dialecte. Le catalogue distingue donc stable, bêta et expérimental. L'industrialisation demanderait une matrice de conformité exécutée pour chaque version de moteur et une surveillance des coûts de requête.

Le deuxième risque concerne la détection. Les seuils et modèles utilisent un historique limité ; ils ne disposent pas d'un jeu annoté permettant de mesurer précision, rappel et taux de faux positifs. Le troisième concerne la narration : la validation de schéma garantit une structure, pas l'exactitude des hypothèses. Un mécanisme de retour humain, une évaluation par scénario et des garde-fous sur les données transmises restent nécessaires.

Enfin, la preuve locale ne couvre ni montée en charge multi-région, ni reprise après sinistre, ni engagement de disponibilité, ni conformité réglementaire. Le mode observe-only de la gouvernance IA est intentionnel : il rend des lacunes visibles, mais ne bloque pas une release. Ces limites n'invalident pas le prototype ; elles définissent précisément le travail requis pour passer d'un PFE démontrable à un service commercial fiable.

Les résultats sont des preuves locales et ne constituent pas une garantie de montée en charge, de disponibilité internet ou de délai de fournisseur LLM. PostgreSQL est le connecteur stable de référence ; DuckDB et SQLite restent bêta, tandis que les autres connecteurs implémentés sont expérimentaux ou soumis à des validations d'environnement. Les résultats de la narration IA doivent être relus car ils peuvent formuler une hypothèse plausible mais non prouvée. La gouvernance IA reste une preuve de concept centrée sur un système seedé : elle observe l'état de quelques preuves, ne bloque pas l'exécution, ne vérifie ni biais ni robustesse et ne certifie aucune conformité.

4.7 Conclusion

La réalisation met en évidence une chaîne cohérente de l'acquisition du signal jusqu'à l'alerte et à l'assistance à l'investigation. La validation, les mesures et les limites sont documentées afin que la démonstration reste honnête et reproductible.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Cadre du travail

Ce projet a permis de concevoir et réaliser DataWatch, une plateforme de surveillance de la qualité des données destinée à un contexte SaaS multi-tenant. Sa chaîne technique relie variété des sources, profilage agrégé, exécution asynchrone, détection hybride, incidents, alertes et narration IA. Elle traite ainsi les contraintes de volume, vélocité et véracité sans déplacer les données métier ligne par ligne. La conception privilégie le cloisonnement des organisations, la traçabilité, l'explicabilité et la validation reproductible.

Synthèse des apports

Sur le plan technique, le projet relie les quatre contraintes classiques d'un système data — volume, variété, vélocité et véracité — à une chaîne concrète : connecteurs, agrégats, file de tâches, profils, détecteurs, incidents, narration et alertes. L'apport IA n'est donc pas un écran ajouté après coup. Il réside dans la comparaison de méthodes déterministes, statistiques, non supervisées et temporelles, puis dans une explication générative tenue à distance de la décision. Sur le plan méthodologique, capacités déclarées, états explicites, révisions immuables et résultats reproductibles forment une discipline de preuve. Sur le plan utilisateur, des métriques éparses deviennent un parcours d'investigation.

La révision finale a démarré et seedé la pile Docker, rejoué cinq parcours navigateur et vérifié l'incident P1 orders. Avec la racine du dépôt montée dans l'image de service, la suite backend atteint 323 réussites et 50 tests ignorés. Le rejeu hôte a aussi exposé des dépendances optionnelles absentes et l'incompatibilité de Python 3.14. Ce sont des preuves locales : ni SLA, ni test de charge, ni certification.

Perspectives

- Mener des benchmarks répétés par connecteur, volume, largeur de schéma et niveau de concurrence.
- Évaluer précision, rappel et faux positifs des détecteurs sur des jeux de données annotés.
- Faire évoluer la gouvernance IA du prototype observe-only vers un parcours configurable : versions, usages, manifestes, approbations, gates de release, exports d'audit et rétention maîtrisée.
- Étendre les validations verticales des connecteurs cloud avec des identifiants et coûts contrôlés.
- Mettre en place des mécanismes de feedback humain sur les narrations IA et leurs hypothèses.

RÉFÉRENCES

- [1] DataWatch, Documentation d'architecture du projet, dépôt source, consultée en août 2026.
- [2] DataWatch, Release verification baseline — 2026-08-21, dossier docs/evidence du projet.
- [3] DataWatch, AI governance phase-two evidence — 2026-08-21, dossier docs/evidence du projet.
- [4] FastAPI, Documentation officielle. <https://fastapi.tiangolo.com/>
- [5] PostgreSQL Global Development Group, Documentation PostgreSQL. <https://www.postgresql.org/docs/>
- [6] React, Documentation officielle. <https://react.dev/>
- [7] Celery Project, Documentation officielle. <https://docs.celeryq.dev/>
- [8] Docker, Documentation officielle. <https://docs.docker.com/>
- [9] ISO/IEC 25012:2008, Software engineering — SQuaRE — Data quality model, version confirmée en 2025.

ANNEXES

Annexe A — Procédure de démonstration

Démarrer la pile Docker et exécuter le seed déterministe.

1. Se connecter à l'espace Acme et ouvrir Opérations.
2. Ouvrir l'incident P1 orders, puis présenter les signaux, la narration et les actions.
3. Afficher le détail de la table et la route d'alerte, puis MailHog.
4. Présenter la gouvernance IA en précisant son périmètre observe-only.

Annexe B — Indicateurs à interpréter avec prudence

Les p50 et p95 mentionnés dans ce rapport ont été obtenus sur une pile Docker locale avec requêtes séquentielles. Ils servent à comparer des régressions dans un même environnement ; ils ne doivent pas être présentés comme des engagements de production. De même, une narration LLM validée atteste de la conformité du format de sortie dans un essai, mais pas d'une exactitude universelle ni d'une latence garantie.

Annexe C — Guide de captures d'écran pour la soutenance

Cette annexe précise les captures à intégrer lorsque la pile de démonstration est démarrée. Chaque image doit être lisible, sans données sensibles, et accompagnée d'une légende expliquant le point vérifié.

C.1 Tableau de bord Opérations

Capture attendue : file des incidents, actifs surveillés et état de santé des sources. La légende doit préciser la priorité de l'incident choisi.

C.2 Détail de l'incident orders

Capture attendue : signaux déclencheurs, sévérité P1/P2, narration IA et actions proposées. La légende doit distinguer une hypothèse générée d'un fait observé.

C.3 Routage d'alerte et gouvernance IA

Capture attendue : route MailHog ou canal configuré, puis écran de gouvernance en mode observe-only. La légende doit expliciter que cette gouvernance ne certifie pas la conformité.

Annexe D — Dictionnaire du modèle de données

Cette annexe décrit les 29 tables applicatives réellement chargées par SQLAlchemy. La table technique alembic_version est exclue. Chaque ligne provient des métadonnées du code : aucun champ n'a été reconstitué à la main. Les références multiples matérialisent les contraintes composites employées pour maintenir l'isolation entre organisations.

Tableau D.1 — Structure de ai_approvals — Avis humains append-only sur un manifeste

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	ai_release_manifests.org_id, users.org_id
system_id	UUID	obligatoire, unicité composée	ai_release_manifests.system_id
manifest_id	UUID	obligatoire	ai_release_manifests.id
reviewer_id	UUID	obligatoire	users.id
reviewer_role	VARCHAR(64)	obligatoire	—
decision	VARCHAR(32)	obligatoire	—
rationale	TEXT	obligatoire	—
evidence_snapshot_hash	VARCHAR(64)	obligatoire	—
evidence_class	VARCHAR(32)	obligatoire	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.2 — Structure de ai_control_evaluations — Résultats typés des contrôles de gouvernance

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	ai_data_use_revisions.org_id, ai_deployments.org_id, ai_evidence.org_id, ai_release_manifests.org_id
system_id	UUID	obligatoire, unicité composée	ai_data_use_revisions.system_id, ai_deployments.system_id, ai_evidence.system_id, ai_release_manifests.system_id
deployment_id	UUID	obligatoire	ai_deployments.id

Colonne	Type	Contraintes	Référence(s)
manifest_id	UUID	obligatoire	ai_release_manifests.id
data_use_revision_id	UUID	facultatif	ai_data_use_revisions.id
evidence_id	UUID	facultatif	ai_evidence.id
control_id	VARCHAR(80)	obligatoire	—
status	VARCHAR(24)	obligatoire	—
evidence_class	VARCHAR(32)	obligatoire	—
observed	JSONB	obligatoire	—
expected	JSONB	obligatoire	—
reason_code	VARCHAR(80)	obligatoire	—
evaluator_version	VARCHAR(64)	obligatoire	—
input_hash	VARCHAR(64)	obligatoire	—
idempotency_key	VARCHAR(128)	obligatoire, unicité composée	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.3 — Structure de ai_data_use_revisions — Déclarations versionnées des usages de données

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	ai_system_versions.org_id, data_sources.org_id
system_id	UUID	obligatoire, unicité composée	ai_system_versions.system_id
version_id	UUID	obligatoire, unicité composée	ai_system_versions.id
source_id	UUID	obligatoire	data_sources.id, monitored_tables.source_id
table_id	UUID	obligatoire	monitored_tables.id
ordinal	INTEGER	obligatoire, unicité composée	—
use_kind	VARCHAR(24)	obligatoire	—
fields	JSONB	obligatoire	—
purpose	TEXT	obligatoire	—

Colonne	Type	Contraintes	Référence(s)
necessity	TEXT	obligatoire	—
steward	VARCHAR(255)	obligatoire	—
sensitivity_ceiling	VARCHAR(32)	obligatoire	—
retention_days	INTEGER	facultatif	—
residency	JSONB	obligatoire	—
allowed_transformations	JSONB	obligatoire	—
expected_db_roles	JSONB	obligatoire	—
vector_contract	JSONB	facultatif	—
schema_fingerprint	VARCHAR(64)	obligatoire	—
canonical_definition	JSONB	obligatoire	—
definition_hash	VARCHAR(64)	obligatoire	—
evidence_class	VARCHAR(32)	obligatoire	—
change_rationale	TEXT	obligatoire	—
created_by	UUID	facultatif	users.id
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.4 — Structure de ai_deployments — Déploiements et manifeste actif

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	ai_release_manifests.org_id, ai_systems.org_id
system_id	UUID	obligatoire, unicité composée	ai_release_manifests.system_id, ai_systems.id
environment	VARCHAR(32)	obligatoire, unicité composée	—
region	VARCHAR(64)	obligatoire, unicité composée	—
workload_identity_hash	VARCHAR(128)	facultatif	—
status	VARCHAR(24)	obligatoire	—
active_manifest_id	UUID	facultatif	ai_release_manifests.id

Colonne	Type	Contraintes	Référence(s)
active_manifest_hash	VARCHAR(64)	facultatif	—
activation_generation	INTEGER	obligatoire	—
created_at	DATETIME	obligatoire	—
updated_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.5 — Structure de ai_evidence — Descripteurs de preuves immuables et sans données brutes

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	ai_data_use_revisions.org_id, ai_deployments.org_id, ai_release_manifests.org_id
system_id	UUID	obligatoire, unicité composée	ai_data_use_revisions.system_id, ai_deployments.system_id, ai_release_manifests.system_id
deployment_id	UUID	obligatoire	ai_deployments.id
manifest_id	UUID	obligatoire	ai_release_manifests.id
data_use_revision_id	UUID	facultatif	ai_data_use_revisions.id
source_profile_id	UUID	facultatif	table_profiles.id
evidence_type	VARCHAR(80)	obligatoire	—
evidence_class	VARCHAR(32)	obligatoire	—
producer	VARCHAR(120)	obligatoire	—
descriptor	JSONB	obligatoire	—
provenance	JSONB	obligatoire	—
content_hash	VARCHAR(64)	obligatoire	—
evaluator_version	VARCHAR(64)	obligatoire	—
redaction_class	VARCHAR(32)	obligatoire	—
retention_class	VARCHAR(32)	obligatoire	—
valid_from	DATETIME	obligatoire	—
valid_until	DATETIME	facultatif	—
collected_at	DATETIME	obligatoire	—
idempotency_key	VARCHAR(128)	obligatoire, unicité	—

Colonne	Type	Contraintes	Référence(s)
		composée	
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.6 — Structure de `ai_governance_incidents` — Incidents de gouvernance dédupliqués en observation

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
org_id	UUID	obligatoire	ai_control_evaluations.org_id, ai_deployments.org_id
system_id	UUID	obligatoire	ai_control_evaluations.system_id , ai_deployments.system_id
deployment_id	UUID	obligatoire	ai_deployments.id
evaluation_id	UUID	obligatoire	ai_control_evaluations.id
control_id	VARCHAR(80)	obligatoire	—
dedupe_key	VARCHAR(128)	obligatoire	—
severity	VARCHAR(8)	obligatoire	—
status	VARCHAR(24)	obligatoire	—
title	VARCHAR(500)	obligatoire	—
created_at	DATETIME	obligatoire	—
resolved_at	DATETIME	facultatif	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.7 — Structure de `ai_release_manifests` — Manifestes de mise en production adressés par contenu

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	ai_system_versions.org_id
system_id	UUID	obligatoire, unicité composée	ai_system_versions.system_id
version_id	UUID	obligatoire	ai_system_versions.id
schema_version	VARCHAR(64)	obligatoire	—
canonical_manifest	JSONB	obligatoire	—
manifest_hash	VARCHAR(64)	obligatoire, unicité	—

Colonne	Type	Contraintes	Référence(s)
		composée	
evidence_cutoff	DATETIME	obligatoire	—
created_by	UUID	facultatif	users.id
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.8 — Structure de ai_system_versions — Versions immuables des systèmes IA

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	ai_systems.org_id
system_id	UUID	obligatoire, unicité composée	ai_systems.id
version_number	INTEGER	obligatoire, unicité composée	—
definition	JSONB	obligatoire	—
definition_hash	VARCHAR(64)	obligatoire	—
provider	VARCHAR(120)	facultatif	—
model	VARCHAR(200)	facultatif	—
artifact_hash	VARCHAR(128)	facultatif	—
prompt_config_hash	VARCHAR(128)	facultatif	—
evaluation_suite_hash	VARCHAR(128)	facultatif	—
change_rationale	TEXT	obligatoire	—
created_by	UUID	facultatif	users.id
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.9 — Structure de ai_systems — Registre des systèmes IA et responsabilités

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	ai_system_versions.system_id
org_id	UUID	obligatoire, unicité composée	ai_system_versions.org_id, organizations.id, teams.org_id, users.org_id, users.org_id,

Colonne	Type	Contraintes	Référence(s)
			users.org_id
slug	VARCHAR(100)	obligatoire, unicité composée	—
name	VARCHAR(255)	obligatoire	—
lifecycle_status	VARCHAR(24)	obligatoire	—
intended_purpose	TEXT	obligatoire	—
prohibited_uses	JSONB	obligatoire	—
affected_population	TEXT	facultatif	—
autonomy_level	VARCHAR(32)	obligatoire	—
human_oversight	TEXT	obligatoire	—
business_owner_id	UUID	facultatif	users.id
technical_owner_id	UUID	facultatif	users.id
risk_owner_id	UUID	facultatif	users.id
team_id	UUID	facultatif	teams.id
risk_context	JSONB	obligatoire	—
current_version_id	UUID	facultatif, unicité composée	ai_system_versions.id
created_by	UUID	facultatif	users.id
created_at	DATETIME	obligatoire	—
updated_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.10 — Structure de alert_configs — Routes Slack, e-mail ou PagerDuty

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
org_id	UUID	obligatoire	organizations.id
table_id	UUID	facultatif	monitored_tables.id
channel	VARCHAR(50)	obligatoire	—
config	JSONB	obligatoire	—
is_active	BOOLEAN	obligatoire	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.11 — Structure de `api_keys` — Empreintes des clés d'accès programmatiques

Colonne	Type	Contraintes	Référence(s)
<code>id</code>	UUID	PK, obligatoire	—
<code>org_id</code>	UUID	obligatoire	<code>organizations.id</code>
<code>name</code>	VARCHAR(100)	obligatoire	—
<code>key_hash</code>	VARCHAR(255)	obligatoire	—
<code>created_at</code>	DATETIME	obligatoire	—
<code>last_used_at</code>	DATETIME	facultatif	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.12 — Structure de `check_results` — Résultats unitaires des contrôles de qualité

Colonne	Type	Contraintes	Référence(s)
<code>id</code>	UUID	PK, obligatoire	—
<code>table_id</code>	UUID	obligatoire	<code>monitored_tables.id</code>
<code>profile_id</code>	UUID	facultatif	<code>table_profiles.id</code>
<code>check_type</code>	VARCHAR(100)	obligatoire	—
<code>check_name</code>	VARCHAR(255)	obligatoire	—
<code>column_name</code>	VARCHAR(255)	facultatif	—
<code>status</code>	VARCHAR(50)	obligatoire	—
<code>observed_value</code>	FLOAT	facultatif	—
<code>expected_range</code>	JSONB	facultatif	—
<code>deviation_score</code>	FLOAT	facultatif	—
<code>checked_at</code>	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.13 — Structure de `custom_monitors` — Moniteurs SQL historiques restreints

Colonne	Type	Contraintes	Référence(s)
<code>id</code>	UUID	PK, obligatoire	—
<code>table_id</code>	UUID	obligatoire	<code>monitored_tables.id</code>
<code>org_id</code>	UUID	obligatoire	<code>organizations.id</code>
<code>name</code>	VARCHAR(255)	obligatoire	—
<code>description</code>	TEXT	facultatif	—
<code>sql_query</code>	TEXT	obligatoire	—

Colonne	Type	Contraintes	Référence(s)
severity	VARCHAR(10)	obligatoire	—
is_active	BOOLEAN	obligatoire	—
run_on_profile	BOOLEAN	obligatoire	—
created_by	UUID	facultatif	users.id
created_at	DATETIME	obligatoire	—
last_run_at	DATETIME	facultatif	—
last_result	JSONB	facultatif	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.14 — Structure de data_sources — Sources de données et configuration chiffrée

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	organizations.id
name	VARCHAR(255)	obligatoire	—
type	VARCHAR(50)	obligatoire	—
connection_config	JSONB	obligatoire	—
status	VARCHAR(50)	obligatoire	—
last_connected_at	DATETIME	facultatif	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.15 — Structure de incidents — Anomalies regroupées et cycle de traitement

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
org_id	UUID	obligatoire	organizations.id
table_id	UUID	obligatoire	monitored_tables.id
severity	VARCHAR(10)	obligatoire	—
status	VARCHAR(50)	obligatoire	—
title	VARCHAR(500)	obligatoire	—
fired_checks	JSONB	facultatif	—

Colonne	Type	Contraintes	Référence(s)
llm_narration	JSONB	facultatif	—
created_at	DATETIME	obligatoire	—
acknowledged_at	DATETIME	facultatif	—
resolved_at	DATETIME	facultatif	—
assignee_id	UUID	facultatif	users.id
assigned_team_id	UUID	facultatif	teams.id
acknowledged_by_id	UUID	facultatif	users.id
resolved_by_id	UUID	facultatif	users.id

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.16 — Structure de invites — Invitations temporaires des membres

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
org_id	UUID	obligatoire	organizations.id
email	VARCHAR(255)	obligatoire	—
role	VARCHAR(50)	obligatoire	—
token	VARCHAR(255)	obligatoire, unicité composée	—
invited_by	UUID	facultatif	—
expires_at	DATETIME	obligatoire	—
accepted_at	DATETIME	facultatif	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.17 — Structure de monitor_evaluation_states — État de brèche, reprise et temporisation

Colonne	Type	Contraintes	Référence(s)
monitor_id	UUID	PK, obligatoire	monitor_revisions.monitor_id, monitor_runs.monitor_id, monitors.id
org_id	UUID	obligatoire	monitors.org_id, organizations.id
revision_id	UUID	obligatoire	monitor_revisions.id
phase	VARCHAR(20)	obligatoire	—
breach_streak	INTEGER	obligatoire	—

Colonne	Type	Contraintes	Référence(s)
recovery_streak	INTEGER	obligatoire	—
cooldown_until	DATETIME	facultatif	—
last_run_id	UUID	facultatif	monitor_runs.id
last_sequence_at	DATETIME	facultatif	—
last_idempotency_key	VARCHAR(128)	facultatif	—
version	INTEGER	obligatoire	—
updated_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.18 — Structure de `monitor_revisions` — Révisions immuables des définitions DSL

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
monitor_id	UUID	obligatoire, unicité composée	monitors.id
revision	INTEGER	obligatoire, unicité composée	—
definition_version	VARCHAR(64)	obligatoire	—
definition_hash	VARCHAR(64)	obligatoire	—
definition	JSONB	obligatoire	—
validation_status	VARCHAR(20)	obligatoire	—
schema_fingerprint	VARCHAR(64)	facultatif	—
created_by	UUID	facultatif	users.id
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.19 — Structure de `monitor_runs` — Exécutions idempotentes liées à une révision

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	monitors.org_id, organizations.id
monitor_id	UUID	obligatoire, unicité composée	monitor_revisions.monitor_id, monitors.id, monitors.id

Colonne	Type	Contraintes	Référence(s)
revision_id	UUID	obligatoire	monitor_revisions.id
table_id	UUID	obligatoire	monitored_tables.id, monitors.table_id
idempotency_key	VARCHAR(128)	obligatoire, unicité composée	—
trigger_type	VARCHAR(20)	obligatoire	—
profile_id	UUID	facultatif	—
sequence_at	DATETIME	obligatoire	—
queued_at	DATETIME	obligatoire	—
plan_hash	VARCHAR(64)	obligatoire	—
planner_version	VARCHAR(64)	obligatoire	—
definition_hash	VARCHAR(64)	obligatoire	—
schema_fingerprint	VARCHAR(64)	facultatif	—
status	VARCHAR(20)	obligatoire	—
attempt	INTEGER	obligatoire	—
claim_token	UUID	facultatif	—
lease_expires_at	DATETIME	facultatif	—
measurements	JSONB	facultatif	—
result	JSONB	facultatif	—
error_code	VARCHAR(64)	facultatif	—
error	TEXT	facultatif	—
started_at	DATETIME	facultatif	—
completed_at	DATETIME	facultatif	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.20 — Structure de `monitored_tables` — Tables inscrites au cycle de surveillance

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
source_id	UUID	obligatoire, unicité composée	data_sources.id
schema_name	VARCHAR(255)	obligatoire, unicité composée	—

Colonne	Type	Contraintes	Référence(s)
table_name	VARCHAR(255)	obligatoire, unicité composée	—
freshness_column	VARCHAR(255)	facultatif	—
check_interval_minutes	INTEGER	obligatoire	—
sensitivity	FLOAT	obligatoire	—
is_active	BOOLEAN	obligatoire	—
dbt_model_yaml	TEXT	facultatif	—
autopilot	JSONB	facultatif	—
check_config	JSONB	facultatif	—
created_at	DATETIME	obligatoire	—
last_profiled_at	DATETIME	facultatif	—
owner_team_id	UUID	facultatif	teams.id
owner_user_id	UUID	facultatif	users.id

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.21 — Structure de monitors — Identité et cycle de vie des moniteurs typés

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	monitor_revisions.monitor_id
org_id	UUID	obligatoire, unicité composée	organizations.id
table_id	UUID	obligatoire, unicité composée	monitored_tables.id
name	VARCHAR(63)	obligatoire, unicité composée	—
mode	VARCHAR(20)	obligatoire	—
status	VARCHAR(20)	obligatoire	—
current_revision	INTEGER	obligatoire	—
active_revision_id	UUID	facultatif	monitor_revisions.id
created_by	UUID	facultatif	users.id
created_at	DATETIME	obligatoire	—
updated_at	DATETIME	obligatoire	—

Colonne	Type	Contraintes	Référence(s)
activated_at	DATETIME	facultatif	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.22 — Structure de oncall_schedules — Créneaux d'astreinte des équipes

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
team_id	UUID	obligatoire	teams.id
user_id	UUID	obligatoire	users.id
starts_at	DATETIME	obligatoire	—
ends_at	DATETIME	obligatoire	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.23 — Structure de organizations — Espaces clients, offre et état d'abonnement

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
name	VARCHAR(255)	obligatoire	—
slug	VARCHAR(100)	obligatoire	—
plan	VARCHAR(50)	obligatoire	—
llm_api_key_encrypted	TEXT	facultatif	—
llm_model	VARCHAR(200)	facultatif	—
stripe_customer_id	VARCHAR(100)	facultatif	—
paypal_subscription_id	VARCHAR(100)	facultatif	—
billing_period	VARCHAR(20)	facultatif	—
subscription_status	VARCHAR(50)	obligatoire	—
trial_ends_at	DATETIME	facultatif	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.24 — Structure de staff_users — Comptes séparés du personnel DataWatch

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—

Colonne	Type	Contraintes	Référence(s)
email	VARCHAR(255)	obligatoire	—
password_hash	VARCHAR(255)	obligatoire	—
full_name	VARCHAR(255)	facultatif	—
is_active	BOOLEAN	obligatoire	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.25 — Structure de table_profiles — Instantanés de profilage et provenance

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
table_id	UUID	obligatoire	monitored_tables.id
collected_at	DATETIME	obligatoire	—
row_count	BIGINT	facultatif	—
freshness_seconds	FLOAT	facultatif	—
schema_fingerprint	VARCHAR(64)	facultatif	—
column_metrics	JSONB	facultatif	—
profile_provenance	JSONB	facultatif	—
profiling_duration_ms	INTEGER	facultatif	—
error	TEXT	facultatif	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.26 — Structure de team_members — Appartenance et rôle d'un utilisateur dans une équipe

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
team_id	UUID	obligatoire	teams.id
user_id	UUID	obligatoire	users.id
role	VARCHAR(50)	obligatoire	—
joined_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.27 — Structure de teams — Équipes opérationnelles d'une organisation

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité	—

Colonne	Type	Contraintes	Référence(s)
		composée	
org_id	UUID	obligatoire, unicité composée	organizations.id
name	VARCHAR(255)	obligatoire	—
description	TEXT	facultatif	—
color	VARCHAR(7)	facultatif	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.28 — Structure de user_notification_prefs — Préférences individuelles de notification

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire	—
user_id	UUID	obligatoire	users.id
org_id	UUID	obligatoire	organizations.id
notify_assigned	BOOLEAN	obligatoire	—
notify_team	BOOLEAN	obligatoire	—
notify_status_change	BOOLEAN	obligatoire	—
daily_digest	BOOLEAN	obligatoire	—
digest_hour	INTEGER	obligatoire	—
mute_until	DATETIME	facultatif	—
created_at	DATETIME	obligatoire	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau D.29 — Structure de users — Comptes humains rattachés à une organisation

Colonne	Type	Contraintes	Référence(s)
id	UUID	PK, obligatoire, unicité composée	—
org_id	UUID	obligatoire, unicité composée	organizations.id
email	VARCHAR(255)	obligatoire	—
password_hash	VARCHAR(255)	obligatoire	—
role	VARCHAR(50)	obligatoire	—
full_name	VARCHAR(255)	facultatif	—

Colonne	Type	Contraintes	Référence(s)
is_active	BOOLEAN	obligatoire	—
created_at	DATETIME	obligatoire	—
last_login_at	DATETIME	facultatif	—

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Annexe E — État du jeu de démonstration

Le seed contrôlé contient 922 lignes applicatives. Sa densité n'est pas uniforme : 463 profils et 376 résultats de contrôle donnent de la matière aux courbes, tandis que les entités de configuration restent volontairement peu nombreuses. Les extraits ci-dessous retirent mots de passe, clés, jetons et configurations chiffrées.

Tableau E.1 — Effectifs du jeu de démonstration

Table	Lignes	Lecture
table_profiles	463	historique dense
check_results	376	historique dense
ai_control_evaluations	12	scénario présent
ai_evidence	12	scénario présent
incidents	7	scénario présent
monitored_tables	7	scénario présent
team_members	7	scénario présent
oncall_schedules	6	scénario présent
ai_governance_incidents	5	scénario présent
teams	5	scénario présent
user_notification_prefs	5	scénario présent
users	5	scénario présent
data_sources	2	scénario présent
organizations	2	scénario présent
ai_approvals	1	scénario présent
ai_data_use_revisions	1	scénario présent
ai_deployments	1	scénario présent
ai_release_manifests	1	scénario présent
ai_system_versions	1	scénario présent
ai_systems	1	scénario présent
alert_configs	1	scénario présent
staff_users	1	scénario présent
api_keys	0	structure prête, non seedée

Table	Lignes	Lecture
custom_monitors	0	structure prête, non seedée
invites	0	structure prête, non seedée
monitor_evaluation_states	0	structure prête, non seedée
monitor_revisions	0	structure prête, non seedée
monitor_runs	0	structure prête, non seedée
monitors	0	structure prête, non seedée

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau E.2 — Organisations de démonstration

name	plan	slug	subscription_status
Acme Corp	growth	acme-corp	active
Startup.io	growth	startup-io	active

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau E.3 — Utilisateurs Acme sans identifiants sensibles

role	full_name	is_active
admin	Alice Chen	True
member	Bob Martin	True
owner	None	True

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau E.4 — Équipes Acme

name	description
Analytics	Business intelligence and reporting
Data Engineering	Owns all data pipelines and warehouse integrity
Platform	Infrastructure and developer tooling

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau E.5 — Sources Acme

name	type	status
Acme Shop DB (live)	postgres	connected

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau E.6 — Tables surveillées chez Acme

table	active	schema	sensitivity	interval_minutes
events	True	public	3	5
orders	True	public	3	5
products	True	public	3	15
users	True	public	3	10

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau E.7 — Incidents Acme ouverts

title	status	severity
[P1] events — freshness SLA breached	open	P1
[P1] orders — payment_status null rate spiked and freshness breach	open	P1
[P1] products — freshness SLA breached	open	P1
[P1] users — freshness SLA breached	open	P1

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Tableau E.8 — Systèmes IA déclarés

name	autonomy_level	lifecycle_status
Support knowledge assistant	assistive	production

Source : élaboration personnelle à partir de la conception et des validations de DataWatch.

Annexe F — Commandes de reproduction

Les commandes suivantes reconstruisent la pile, réinitialisent les données, réexportent la preuve du schéma et régénèrent les 24 vues du rapport. Les identifiants de démonstration restent dans le guide local ; ils ne constituent pas des secrets de production.

```
docker compose up -d --wait
docker compose --profile seed run --rm --entrypoint python seed
/scripts/quickstart.py --reset
curl -fsS http://localhost:8000/ready
python scripts/pfe/export_database_evidence.py
python scripts/pfe/generate_report_visuals.py
cd frontend && npm run capture:pfe
```